Online supplement: A Comparability Protocol for Benchmarking Relational Database Access Layers in Express.js

Mateusz Miotk
Faculty of Mathematics, Physics and Informatics,
University of Gdańsk
mateusz.miotk@ug.edu.pl

This supplement carries measurement detail referenced from the main text. The complete revision artifact is archived as release v1.13.17, which the concept DOI https://doi.org/10.5281/zenodo.21313858 resolves to; the Zenodo record also carries an immutable per-version DOI for that release. This release adds the specification-derived oracle, campaign-state preflight, a replacement corrected-state primary campaign, a second same-host RQ2 validation campaign, fresh secondary measurements, the external-audit dataset, and blind review packets. The revised primary tables are generated from the corrected-state campaign rather than inherited from the earlier baseline.

AI-assisted research tooling. The harness, table generators, and statistical estimators behind these tables were implemented with assistance from Claude (Anthropic), which also proposed candidate analysis procedures that the author selected among, verified, and interpreted. The models, affected components, the checks that independently validate the generated code, and the limits of those checks are documented in the main text (Study Design, "AI-assisted research tooling"). Every table below reporting a measurement is regenerated from archived raw records by a committed generator listed in `MANIFEST.md`; `MANIFEST.md` marks the authored analytical tables, which report no measurement.

1 Dispersion on MySQL

Table S1 reports run-level throughput dispersion on MySQL. All five patterns use the one corrected-state campaign. Figure S1 foregrounds all corrected insert replicates because a median and CV can hide mixture or tail structure. Historical insert samples from the defective reset are preserved in the archive but are not used for the revised ordering.

Table S1: Coefficient of variation (%) of throughput across the 25 repeated runs, per access layer and pattern on MySQL. Low values indicate stable measurements. The two tuned baselines are excluded: they are disclosed reference points rather than compared treatments. Their insert cells are the most dispersed in the campaign (`pg-tuned` 8.9%, `mysql2-tuned` 12.8%), so the bounds below describe the compared set, not every measured cell.

Layer	**point_read**	**range_scan**	**deep_fetch**	**aggregation**	**write**
mysql2	3.2	2.1	1.0	2.8	13.6
knex	1.8	2.4	2.4	2.4	13.6
drizzle	1.4	1.5	2.8	2.2	11.8
prisma	2.8	1.8	1.9	2.0	5.1
sequelize	3.6	2.0	1.6	4.4	10.8
typeorm	2.3	3.6	1.9	2.0	7.5
objection	2.6	2.6	3.0	3.6	11.9
mikroorm	3.2	3.2	3.7	3.0	8.8

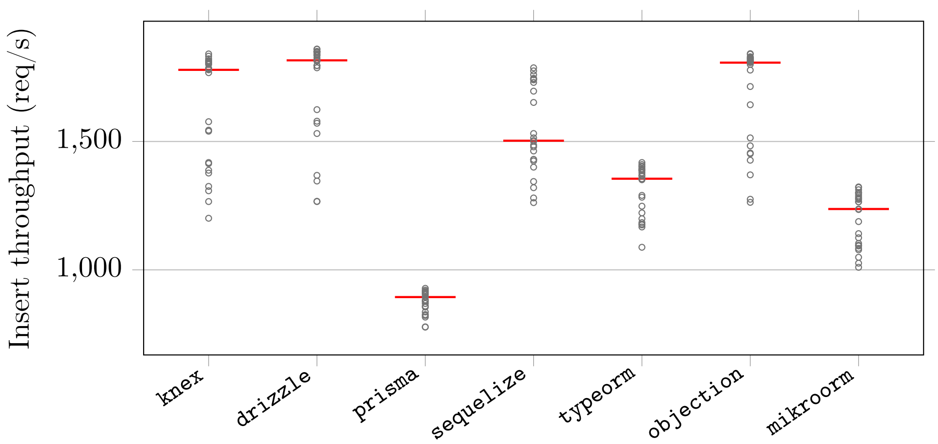

Figure S1: Corrected-state raw per-replicate MySQL insert throughput for the seven portable layers (25 independent repetitions; red bar = median; maximum CV 13.6%). The points, rather than only medians and intervals, expose any mixture or tail structure. Historical insert samples affected by the reset defect are not used for the corrected ordering.

2 Policy-selected documented round-trip counts

Table S2 reports, for each layer, the number of SQL statements the policy-selected documented deep fetch issues per request, captured from server-side statement logs. The counts motivate the main text's observation that round-trip count alone does not determine throughput: on the PostgreSQL deep fetch single-query MikroORM is slowest, while two-query Drizzle outruns four-query Prisma.

Table S2: Number of SQL round trips each access layer issues per request on PostgreSQL, captured from server-side statement logging. The native drivers, Knex, and Drizzle issue a two-query deep fetch; the data-mapper ORMs' eager-loading strategies choose their own counts. Full generated SQL and `EXPLAIN` plans are in the replication package.

Layer	Point read	Range scan	Deep fetch	Aggregation	Insert
`pg`	1	1	2	1	1
`knex`	1	1	2	1	1
`drizzle`	1	1	2	1	1
`prisma`	1	1	4	1	1
`sequelize`	1	1	1	1	1
`typeorm`	1	1	2	1	2
`objection`	1	1	4	1	1
`mikroorm`	1	1	1	1	1

3 Single-row insert throughput under default versus relaxed durability

Table S3 gives the per-layer insert throughput under each engine's default durability configuration (the paper's primary single-row-insert regime) and under the symmetric relaxed regime (secondary, 5 replicates), with the relaxed-to-default ratio quoted in the main text.

4 Equal-CPU-budget sensitivity check

Table S4 gives the deep-fetch throughput with the server confined to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). The main text states the equal-CPU check as an operating condition; the nine values are reported here.

Table S3: Insert throughput (req/s) under the default durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`; median of 25 replicates) and under the relaxed regime (all of the above off; median of 5), with the relaxed÷default ratio. Ratios are reported per layer and backend stack; this sensitivity changes durability settings and therefore does not estimate library-intrinsic overhead.

	PostgreSQL			MySQL		
Layer	default	relaxed	ratio	default	relaxed	ratio
`pg`	6018	6883	1.14	–	–	–
`mysql2`	–	–	–	1766	4278	2.42
`pg-tuned`	6125	6937	1.13	–	–	–
`mysql2-tuned`	–	–	–	1683	4113	2.44
`knex`	4581	5196	1.13	1779	3676	2.07
`drizzle`	4091	4331	1.06	1816	3310	1.82
`prisma`	2695	2637	0.98	894	911	1.02
`sequelize`	2616	2541	0.97	1503	2222	1.48
`typeorm`	2567	2579	1.00	1355	1608	1.19
`objection`	3780	4038	1.07	1807	3139	1.74
`mikroorm`	1894	1837	0.97	1237	1325	1.07

Table S4: Exploratory equal-CPU sensitivity check (deep-fetch throughput, req/s, PostgreSQL, median of 3 runs, three selected layers) with the server process confined by `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). This bounded sensitivity exposes whether the displayed deep-fetch ordering changes when the application process receives the same 1-, 2-, or 4-core budget. Database and generator resources remain shared-host controls, not a cross-machine comparison.

Layer	1 core	2 cores	4 cores
`pg`	3689	3688	3687
`prisma`	1079	1052	1219
`mikroorm`	446	495	522

5 Application-tier versus end-to-end CPU efficiency

Table S5 reports, per layer on the deep fetch, the median application-process-tree CPU and database-server CPU, the combined core-count, and throughput per application-CPU-second and per combined (application+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database, but on the current versions the application-only and combined figures largely agree, because no layer draws more than about one application core: `mysql2` leads application-tier efficiency by about 3.8× over Prisma (2,301 vs. 600 req per app-CPU-s) and by 3.4× end-to-end (1,272 vs. 371 req per combined-CPU-s). There is no CPU-for-throughput trade of the kind an earlier version of this study reported.

Table S5: Application-tier versus end-to-end CPU efficiency on the deep fetch (MySQL): median CPU (% of one logical core) of the Node.js process tree and of the database server, the combined core-count, and throughput per application-CPU-second and per combined (app+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database; the combined figure is the end-to-end view. Load-generator CPU is excluded.

Layer	app CPU%	db CPU%	comb. cores	req/app-CPU-s	req/comb-CPU-s
mysql2	105	85	1.90	2301	1272
knex	100	75	1.75	1982	1133
drizzle	105	50	1.55	1239	839
prisma	105	65	1.70	600	371
sequelize	100	30	1.30	880	677
typeorm	100	70	1.70	1017	598
objection	100	45	1.45	836	577
mikroorm	110	25	1.35	429	350

6 Coordinated-omission-corrected open-loop sweep

Table S6 gives the full constant-arrival sweep (PostgreSQL, five layers, offered rates 250–4,000 requests/s) behind the open-loop paragraph of the main text: achieved throughput, corrected latency percentiles measured from each request's *intended* send time, and timeout counts (10 s cap, clipped into the distribution).

7 Pool-size sensitivity

The primary campaign fixes every layer's connection pool at 10 to isolate the access layer from pool tuning. Table S7 varies the pool from 1 to 50 for one

Table S6: Open-loop tail latency, corrected for coordinated omission: p99 (ms) on the deep fetch (PostgreSQL) under a constant offered request rate, with every latency measured from the request's intended start time and timed-out requests included at their clipped value. † marks cells that did not sustain the offered rate (achieved<95% or any timeouts).

Layer	250/s	500/s	1000/s	2000/s	4000/s
`pg`	2	2	2	3	1483†
`prisma`	3	3	13	10721†	10947†
`knex`	2	2	2	89	8812†
`sequelize`	3	3	183	10745†	10940†
`mikroorm`	4	346	11283†	11677†	11935†

layer of each tier on the deep fetch (PostgreSQL, 50 connections). The deep-fetch ordering (`pg` > Knex > Prisma > MikroORM) holds at every pool size of two or more, and each layer's throughput saturates at a pool well below 50, so the fixed-pool choice does not appear to drive the ranking. A pool of one is the exception and the one regime that compresses the differences, because every layer then queues on connection acquisition: there Prisma (870) overtakes Knex (741) on PostgreSQL.

Table S7: Deep-fetch throughput (req/s, PostgreSQL, 50 connections, median of 3) as the connection pool varies from 1 to 50, per access layer. The primary campaign fixes the pool at 10; the displayed frontier shows the sensitivity of each representative implementation to that shared choice.

Layer	1	2	5	10	20	50
`pg`	905	1762	3224	3667	3798	3768
`knex`	741	1413	2392	2532	2683	2636
`prisma`	870	1072	1183	1234	1207	1105
`mikroorm`	494	487	507	527	527	522

8 Transactional multi-statement write

Beyond the single-row insert of the primary matrix, Table S8 reports a transactional write, `POST /threads`, which inserts a post and five comments in one transaction through each layer's transaction facility, for one layer of each tier on PostgreSQL under default durability (median of five runs, exploratory). The full layer × engine matrix for this pattern is left to future work. The transaction commits once; its tighter clustering than the widest read spread is consistent with amortizing per-commit work (3.7×, `pg` 2,718 down to Prisma 733), though the ordering does not simply track the single-row insert: Prisma, mid-pack on the single-row PostgreSQL insert, falls to the bottom of the transactional subset.

Table S8: Transactional write throughput (req/s) and tail latency (p99, ms) for `POST /threads`, which inserts a post and five comments in a single transaction, on PostgreSQL under default durability (50 connections, median of 5 runs). One layer of each tier is shown; the full matrix is outside this sensitivity check. Across the declared subset, the observed throughput spread is 3.71×. This exploratory PostgreSQL subset does not support a general claim about writes.

Layer	req/s	p99 (ms)
`pg`	2718	22
`knex`	2322	27
`prisma`	733	80
`typeorm`	1935	32
`mikroorm`	856	68

9 Dispersion on PostgreSQL

Table S9 gives the full per-layer, per-pattern coefficient of variation of throughput on PostgreSQL across the 25 repeated runs (maximum 6.4%, `pg`/insert), the companion to the MySQL table in Section S1.

Table S9: Coefficient of variation (%) of throughput across the 25 repeated runs, per access layer and pattern on PostgreSQL. Low values indicate stable measurements. The two tuned baselines are excluded: they are disclosed reference points rather than compared treatments. Their insert cells are the most dispersed in the campaign (`pg-tuned` 8.9%, `mysql2-tuned` 12.8%), so the bounds below describe the compared set, not every measured cell.

Layer	**point_read**	**range_scan**	**deep_fetch**	**aggregation**	**write**
pg	3.6	2.7	2.6	3.8	6.4
knex	2.6	1.6	2.7	4.6	5.6
drizzle	2.3	2.3	2.3	3.9	4.0
prisma	2.9	2.8	3.0	1.9	3.8
sequelize	3.4	2.6	1.7	5.1	3.5
typeorm	4.4	1.6	2.2	5.1	3.1
objection	3.6	2.8	2.7	4.3	4.1
mikroorm	3.9	3.2	4.5	2.2	6.2

10 Resource footprint

Table S10 gives the per-layer application-process CPU (median % of one logical core) and peak resident memory on the deep fetch (MySQL), the resource-accounting detail supporting the main text's operating conditions (all layers ≈ 100–110% of one core; peak RSS 223–356 MB).

Table S10: Server-process resource footprint on the deep fetch (MySQL): median CPU utilization (% of one core) and peak resident memory (MB) of the Node.js process under load. The load generator and database run in separate processes.

Layer	Category	CPU (%)	RSS (MB)
mysql2	native-driver	105	290
knex	query-builder	100	225
drizzle	orm	105	298
prisma	orm	105	334
sequelize	orm	100	259
typeorm	orm	100	226
objection	orm	100	223
mikroorm	orm	110	356

11 Pinned versions

Table S11 lists the pinned versions of every evaluated access layer and tool, recorded from the lockfile on the measurement date.

Table S11: Pinned versions of the evaluated access layers and tooling.

Layer / tool	Version	Layer / tool	Version
`pg`	8.22.0	`sequelize`	6.37.8
`mysql2`	3.23.0	`typeorm`	1.1.0
`knex`	3.3.0	`objection`	3.1.5
`drizzle-orm`	0.45.2	`@mikro-orm/core`	7.1.6
`@prisma/client`	7.8.0	`express`	5.2.1
`autocannon`	8.0.0	Node.js	24.18.0
PostgreSQL	18.4	MySQL	9.7.1

Pinned to the latest stable release compatible with the harness adapter contract at the freeze date (2026-07-15); Sequelize's version-7 line is a prerelease, so the 6.x stable line is used. Prisma 7 is Rust-free and connects through an official JS driver adapter (`@prisma/adapter-pg` for PostgreSQL, `@prisma/adapter-mariadb` with the `mariadb` 3.5.3 driver for MySQL, as Prisma ships no `mysql2` adapter).

12 Per-pattern detail: the read patterns

The two cheapest read patterns show the smallest between-layer spread and are summarized in the main text; their full throughput and p99 tables are given here. Table S12 reports the point read (primary-key lookup) and Table S13 the keyset range scan, each by access layer and engine. The two patterns foregrounded in the analysis (deep fetch and insert) are tabulated in the main text; aggregation is reported in full below.

Table S12: Point read: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by $\leq$ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6713 [6665–6748]	10 [10–10]	–	–
mysql2	native-driver	–	–	4296 [4248–4366]	16 [16–16]
pg-tuned	native-tuned	6708 [6667–6820]	10 [10–10]	–	–
mysql2-tuned	native-tuned	–	–	4278 [4226–4312]	14 [14–15]
knex	query-builder	4909 [4843–4939]	12 [12–13]	3723 [3699–3759]	16 [16–16]
drizzle	orm	4121 [4069–4162]	14 [14–14]	2854 [2828–2874]	22 [22–23]
prisma	orm	3219 [3207–3243]	20 [20–21]	2027 [2019–2041]	29 [29–30]
sequelize	orm	3466 [3383–3507]	17 [16–18]	2680 [2627–2709]	22 [21–23]
typeorm	orm	4168 [4091–4184]	15 [15–15]	2961 [2945–2998]	19 [19–20]
objection	orm	3927 [3863–3964]	15 [15–16]	3146 [3123–3168]	18 [18–19]
mikroorm	orm	2373 [2338–2396]	25 [25–27]	2004 [1997–2043]	28 [27–29]

13 Common-SQL raw-path sensitivity contrast: full table

Table S14 gives the full common-SQL raw-path sensitivity contrast on the deep fetch for both engines: throughput when every layer executes identical statements verified by server-side capture. The smaller residual spread is a compound sensitivity contrast, not a bound or causal decomposition.

14 Per-pattern detail: aggregation

Table S15 gives the full policy-selected aggregation-pattern throughput and p99 by access layer and engine. RQ3 finds aggregation the least layer-sensitive of the five primary patterns; the shared logical query avoids treating

Table S13: Keyset range scan: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	3808 [3764–3832]	18 [17–18]	–	–
mysql2	native-driver	–	–	3293 [3260–3344]	21 [21–21]
pg-tuned	native-tuned	3798 [3744–3820]	17 [17–18]	–	–
mysql2-tuned	native-tuned	–	–	3257 [3201–3292]	18 [18–19]
knex	query-builder	3130 [3117–3157]	19 [19–20]	2958 [2938–2992]	20 [19–21]
drizzle	orm	2746 [2724–2771]	21 [20–22]	2145 [2134–2163]	29 [29–30]
prisma	orm	2422 [2392–2446]	26 [26–27]	1526 [1519–1538]	39 [38–40]
sequelize	orm	2428 [2391–2445]	24 [24–25]	2082 [2064–2104]	27 [26–28]
typeorm	orm	2555 [2538–2572]	24 [23–25]	2217 [2177–2251]	26 [26–27]
objection	orm	2660 [2633–2676]	22 [22–23]	2534 [2503–2544]	23 [22–24]
mikroorm	orm	903 [896–909]	65 [63–67]	848 [839–856]	66 [65–73]

differing aggregate semantics as a performance result, but does not isolate mapping or protocol costs.

15 Policy-selected documented deep-fetch choices per access layer

Table S16 documents, for each access layer, the exact eager-loading API used on the deep fetch, the number of SQL round trips it issues, the documented alternative loading strategy we did *not* use, and the query-logging state. Every layer uses the path selected by the frozen-page policy; query logging is disabled and verified on all eleven adapters, and no lifecycle hooks or validation run on the read path. The six data-mapper and ORM layers split both ways on the join-versus-select-in selected path: Sequelize, TypeORM, and MikroORM use a single selected join, while Prisma and Objection use selected separate batched queries. The loading-strategy sensitivity check reported in the main text (Table S18) switches the join-selected layers to select-in where that alternative is a byte-identical drop-in. Full per-adapter detail is in the replication package's `METHODOLOGY.md`.

Table S14: Common-SQL raw-path sensitivity — the residual same-SQL spread on the deep fetch: throughput (req/s) of the policy-selected documented deep fetch versus the *same-SQL* contrast, in which every layer executes the identical two-statement SQL with identical row mapping through its raw-SQL facility. The two columns come from different campaigns: the selected column is the median of the 25-run primary campaign, the same-SQL column the median of a separate 10-run campaign. The per-layer ratios are therefore non-contemporaneous descriptive contrasts rather than paired within-campaign estimates, and carry no interval. The ratio (selected ÷ same-SQL) measures the gap between each layer's policy-selected documented relational-fetch path and the common raw-SQL path, a composite of query strategy, query count, protocol, the raw versus ORM API, and result marshalling and hydration changed together; the residual native-relative same-SQL spread (1.71× on PostgreSQL, 2.03× on MySQL) is a common-SQL raw-path contrast on that compound difference, not a bound, and isolates no single factor.

	PostgreSQL			MySQL		
Layer	selected	same-SQL	ratio	selected	same-SQL	ratio
`pg`	3638	3654	1.00	–	–	–
`mysql2`	–	–	–	2416	2459	0.98
`knex`	2431	2691	0.90	1982	2229	0.89
`drizzle`	1829	3371	0.54	1301	2077	0.63
`prisma`	1175	2136	0.55	630	1212	0.52
`sequelize`	978	2487	0.39	880	1861	0.47
`typeorm`	1219	3199	0.38	1017	2231	0.46
`objection`	1017	2854	0.36	836	2263	0.37
`mikroorm`	519	3212	0.16	472	2395	0.20

Table S15: Aggregation: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by $\leq$ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6807 [6703–6889]	10 [9–10]	–	–
mysql2	native-driver	–	–	4450 [4420–4480]	16 [15–16]
pg-tuned	native-tuned	7583 [7460–7609]	9 [9–9]	–	–
mysql2-tuned	native-tuned	–	–	4398 [4368–4433]	14 [14–15]
knex	query-builder	5577 [5464–5654]	12 [11–13]	3979 [3955–4022]	16 [15–17]
drizzle	orm	6410 [6288–6434]	10 [10–11]	3963 [3932–3999]	18 [17–18]
prisma	orm	4345 [4269–4382]	15 [15–16]	2379 [2354–2398]	27 [27–28]
sequelize	orm	4625 [4578–4730]	14 [13–15]	3411 [3368–3434]	19 [18–19]
typeorm	orm	6072 [5930–6115]	12 [11–12]	3882 [3853–3911]	16 [15–17]
objection	orm	3736 [3707–3790]	17 [16–17]	2950 [2898–2986]	21 [21–22]
mikroorm	orm	5592 [5498–5642]	12 [12–13]	3995 [3954–4031]	16 [15–17]

Table S16: Policy-selected documented deep-fetch implementation per access layer: the eager-loading API actually used, the SQL round trips it issues (measured from server-side statement logging for the portable layers; by construction for the native variants), the documented alternative loading strategy we did *not* use, and the query-logging state. Query logging is disabled everywhere; no lifecycle hooks or validation run on the read path (details in the replication package's `METHODOLOGY.md`).

Layer	Deep-fetch API	Round-trips	Alternative not used	Logging
`pg`	hand-written JOIN ×2	2	n/a (hand-written SQL)	off
`mysql2`	hand-written JOIN ×2	2	n/a (hand-written SQL)	off
`pg-tuned`	named-prepared JOIN ×2	2	n/a (hand-written SQL)	off
`mysql2-tuned`	`execute()` JOIN ×2	2	n/a (hand-written SQL)	off
`knex`	builder `.join()` ×2	2	n/a (hand-written SQL)	off
`drizzle`	builder `.innerJoin()` ×2	2	relational query API (`db.query`)	off
`prisma`	`include:`	4	`relationLoadStrategy: 'join'`	off
`sequelize`	`include: (separate:false)`	1	`separate:true` (select-in)	off
`typeorm`	`relations:`	2	`relationLoadStrategy: 'query'`	off
`objection`	`.withGraphFetched()`	4	`.withGraphJoined()`	off
`mikroorm`	`populate:`	1	`strategy: 'select-in'`	off

16 Open-loop tail latency on MySQL

Table S17 is the MySQL companion to the PostgreSQL open-loop sweep (Supplement Table S6): coordinated-omission-corrected p99 on the deep fetch at a constant offered rate. The sub-saturation tail is flat and small on both engines, and each layer collapses once the offered rate crosses its capacity, so the high-load-tail ordering holds on MySQL as on PostgreSQL.

Table S17: Open-loop tail latency on **MySQL**, corrected for coordinated omission: p99 (ms) on the deep fetch under a constant offered request rate, every latency measured from the intended start time and timed-out requests clipped into the distribution. †marks cells that did not sustain the offered rate. As on PostgreSQL (Supplement Table S6), the sub-saturation tail is flat and small and each layer collapses once the offered rate crosses its capacity, so the high-load-tail ordering is engine-robust.

Layer	250/s	500/s	1000/s	2000/s	4000/s
`mysql2`	2	2	3	15	10856†
`prisma`	5	7	9518†	11723†	11900†
`knex`	2	2	2	243	10904†
`sequelize`	3	3	1628†	10797†	10962†
`mikroorm`	4	953†	11314†	11788†	11912†

17 Alternative eager-loading sensitivity

Table S18 reports the loading-strategy sensitivity check: for the data-mapper ORMs whose alternative strategy is a byte-identical drop-in, policy-selected documented deep-fetch throughput versus the alternative, on both engines. Both endpoints were verified to return byte-identical responses before timing.

18 MySQL insert commit-path wait sensitivity

Table S19 reports `performance_schema` redo-log and binary-log wait measurements around 4,000 valid inserts per displayed layer at default durability. Waits are summed across ten concurrent requests, so aggregate wait per insert may exceed request wall time and the ratio is not a causal time fraction. Similar wait totals and the relaxed-durability control are consistent with a shared commit-path contribution; they neither establish a performance floor nor decompose engine, driver, and adapter costs.

Table S18: Alternative eager-loading sensitivity on the deep fetch: policy-selected documented throughput (req/s) versus the alternative loading strategy, both engines, for the layers whose alternative is a byte-identical drop-in. The table reports only byte-identical drop-ins; excluded alternatives remain configuration-portability findings rather than timed treatments. This is a secondary run that predates the accepted primary campaign, so its selected column sits up to about 14% below the corresponding primary median; only the within-table selected-versus-alternative contrast is interpreted, never the absolute values. On both engines the policy-selected documented path is the *faster* strategy for `sequelize` and `mikroorm`, so their deep-fetch deficit does not appear to be an artifact of a poor selected strategy. The alternative strategy was not a byte-identical drop-in for `typeorm` on PostgreSQL (alt endpoint status 500) and MySQL (alt endpoint status 500); `objection` on PostgreSQL (response not byte-identical), so those cells are omitted; that non-portability is itself a configuration finding rather than a timed result.

		PostgreSQL		MySQL	
Layer	Switch	selected	alt.	selected	alt.
`sequelize`	join→select-in	915	690	830	602
`mikroorm`	join→select-in	447	357	420	324
`objection`	select-in→join	–	–	820	1301

Table S19: MySQL insert commit-path wait-event sensitivity (`performance_schema` wait instruments, default durability `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`): per-insert wall time, the aggregate redo-log+binlog wait per insert, and aggregate wait divided by wall time, over 4000 inserts per layer. Timers sum waits across 10 concurrent requests, so the ratio may exceed 100% and is not a causal time fraction. Similar wait totals across the displayed layers, together with the relaxed-durability contrast (`mysql2`, `trx_commit=0`, `sync_binlog=0`, 3182 req/s), are consistent with a shared commit-path contribution; they do not decompose engine, driver, and adapter costs.

Layer (MySQL)	req/s	per-insert (ms)	aggregate wait/insert (ms)	wait/wall
`mysql2`	1576	0.635	1.099	173%
`knex`	1561	0.641	1.118	175%
`mikroorm`	1041	0.96	1.518	158%
Relaxed durability (control):				
`mysql2`*	3182	0.314	0.011	4%

19 Post-restart environmental-robustness check

Table S20 reports the environmental-robustness check: as the campaign's final stage, after a full restart of both database engines (cold buffer pools, fresh connections), a selected deep-fetch subset was re-measured against the primary median. The three-layer order is unchanged on PostgreSQL (`pg`, Prisma, MikroORM) and MySQL (`mysql2`, Prisma, MikroORM); absolute throughput is up to 16% lower, a cold-buffer and run-to-run drift the paper's relative analysis is designed to absorb. Because PostgreSQL and MySQL cells are not measured simultaneously, this drift is non-uniform across engines (0% on `mysql2`, 16% on `pg`, 16% on MikroORM/PostgreSQL), so the cross-backend ratios remain within-campaign estimates from sequential, randomized cells and can carry residual drift. Broad deep-fetch gaps exceed this check, but close aggregation and insert reversals are not treated as durable transfer evidence; the corrected-state whole-campaign sensitivity is reported separately.

Table S20: Environmental-robustness check (review 6.7): as the campaign's final stage, after a full restart of both database engines (cold buffer pools, fresh connections), deep-fetch throughput of a selected subset was re-measured against the median of the campaign prevailing at that time. That baseline is the superseded pilot: this check ran before the accepted corrected-state campaign, so the pairing is contemporaneous by design rather than an oversight. The ratios report temporal/restart sensitivity for the displayed subset. This is a same-host check and does not estimate cross-machine or deployment transfer.

Engine	Layer	primary req/s	post-restart req/s	ratio
PostgreSQL	`pg`	3687	3100.5	0.84
PostgreSQL	`prisma`	1186	1153	0.97
PostgreSQL	`mikroorm`	516	434.5	0.84
MySQL	`mysql2`	2382	2375.5	1.00
MySQL	`prisma`	634	618.5	0.98
MySQL	`mikroorm`	480	437.5	0.91

20 Utilization-controlled tail latency

Tables S21 and S22 give the coordinated-omission-corrected p99 on the deep fetch when each layer is offered 50/70/85/95% of its $\widehat{C}_{50}$ capacity proxy (the 25-run median 50-connection deep-fetch rate), replicated five times, on both engines. At the stable nominal 50% target the tails contract to 2.3–4.9 ms on PostgreSQL and 1.9–5.5 ms on MySQL. Most 70% cells remain small.

This is consistent with capacity and queueing contributing materially to the wider equal-demand ladder; it neither identifies their share nor establishes a deployment-general sub-saturation bound. Re-expressing the loads against the full-sweep maximum leaves the 50% and 70% bands below saturation (Table S45). This is the capacity-normalized latency companion to the high-load primary p99. The 50% and most 70% cells, which carry that comparison, are stable across runs; the 85/95% cells are increasingly noisy (p99 varies several-fold between runs as a layer approaches saturation), so those columns are read as trend, not precise values.

Table S21: Utilization-controlled open-loop tail on the deep fetch (PostgreSQL): coordinated-omission-corrected p99 (ms) when each layer is offered a fixed fraction of its $\widehat{C}_{50}$ capacity proxy (the 25-run median throughput at 50 connections). Cells give the median of five runs with the observed minimum–maximum in brackets; with only five replicates the bracket is the raw spread, not an interval estimate. The 50% and 70% columns carry the sub-saturation comparison and are interpreted with the denominator sensitivity in Supplement Table S45. The 85% and 95% columns are near-saturation sensitivity evidence, not a precise convergence claim — as their spreads show directly.

Layer	50%	70%	85%	95%
`pg`	2.6 [2.2–3]	4.4 [4–6.7]	5.5 [4.5–10.8]	57.7 [24.6–710.8]
`knex`	2.3 [2.3–2.7]	3.4 [2.5–4.3]	14.1 [6.2–63.6]	53.5 [12.5–484.8]
`drizzle`	2.5 [2.5–2.6]	2.9 [2.8–6.8]	11.7 [8.9–134.1]	54.1 [4.3–293.9]
`prisma`	4.9 [4.7–5.7]	7.8 [6.9–12]	16.2 [15.5–33.5]	56.1 [27.4–314.8]
`sequelize`	2.8 [2.5–3.1]	3.5 [2.8–4.5]	5.8 [4–11.9]	18.8 [15.9–25.5]
`typeorm`	3.5 [3–4]	5.2 [3.6–6.6]	12.9 [10.3–22.9]	46.5 [21.1–85]
`objection`	3 [2.5–3.1]	3.3 [3.1–3.4]	14.2 [12.7–34.2]	42.9 [11.4–103.7]
`mikroorm`	4.1 [4–4.7]	4.6 [4.2–5.3]	7.3 [5.8–15.4]	11.7 [5.9–327]

21 Longer-window tail sensitivity

The primary p99 is measured over a 12 s window, which yields only approximately 60 top-percentile observations in the slowest deep-fetch cells. Table S23 re-measures the same operating point over 60 s (ten runs per cell). The 12-versus-60-second p99 rank correlation is 1.00 on both PostgreSQL and MySQL; the maximum cell-median shift is 5.6% and 3.8%, respectively, and maximum run-level p99 CV is 7.3% and 3.2%. The PostgreSQL difference reflects native-tie resolution as well as endpoint-specific tail-window sensitivity. These data support the broad practical ordering in this campaign, not an exact-invariance claim or a general rule that a 12-second window is sufficient.

Table S22: Utilization-controlled open-loop tail on the deep fetch (MySQL): coordinated-omission-corrected p99 (ms) when each layer is offered a fixed fraction of its $\widehat{C}_{50}$ capacity proxy (the 25-run median throughput at 50 connections). Cells give the median of five runs with the observed minimum–maximum in brackets; with only five replicates the bracket is the raw spread, not an interval estimate. The 50% and 70% columns carry the sub-saturation comparison and are interpreted with the denominator sensitivity in Supplement Table S45. The 85% and 95% columns are near-saturation sensitivity evidence, not a precise convergence claim — as their spreads show directly.

Layer	50%	70%	85%	95%
`mysql2`	4.7 [4–5.6]	7.4 [6.8–7.7]	12.8 [9.5–34]	27.3 [14–68.1]
`knex`	1.9 [1.8–2.2]	1.9 [1.6–3.5]	8.9 [2.3–24.1]	22.7 [8.4–28.3]
`drizzle`	3.9 [3.7–5.2]	5.6 [5.5–6.5]	15.6 [12.6–56]	58 [25.5–263.9]
`prisma`	5.5 [5.3–5.7]	6.6 [6.1–7.1]	9.5 [7.7–11.9]	71.5 [13.4–572.1]
`sequelize`	3.1 [2.4–3.3]	3.3 [3–6.8]	10.1 [4.1–17.8]	11.2 [7.3–16]
`typeorm`	2.8 [2.7–3.2]	3 [3–3.5]	3.3 [2.9–71.1]	17.3 [3–69.5]
`objection`	2.9 [2.6–3.6]	3 [3–3.2]	4.5 [3–10.1]	11.5 [3.2–170.7]
`mikroorm`	4.1 [3.7–4.5]	4.2 [4–4.5]	4.7 [4.2–12]	13.8 [7.5–25.3]

22 Multi-worker scaling

Table S24 scales a selected subset (three layers per engine) to 1, 2, and 4 Express workers (a shared-port `node` cluster), median of three runs. Because each process uses about one core and gains nothing from more, aggregate throughput rises roughly in proportion to workers over this range: on PostgreSQL the native driver leads at one worker (`pg` 3,299 to Prisma's 1,117); its aggregate throughput then plateaus by two workers (4,055, then 3,961 at four) — whether from database contention, shared-host CPU competition, or the fixed-connection generator is not instrumented here — while Prisma, low per process, rises 1,117 to 2,275 to 4,449 and reaches native-like aggregate throughput at four workers. This is a bounded check, not a demonstration of general scalability: it stops at four workers, measures no database-side saturation (DB CPU, connections, or wait events), and on MySQL horizontal scaling *widens* the gap — the native `mysql2` keeps scaling to 8,883 req/s at four workers while Prisma reaches only 2,427 (about `mysql2` at one worker). Within these limits a layer's low per-process throughput may be addressed by additional application processes until another bottleneck is reached, at a proportional cost in cores — a partial test of the single-host resource-competition concern.

Table S23: Longer-window tail sensitivity on the deep fetch. Each cell's primary p99 is measured over a 12 s window (≈60 tail observations per run in the slowest cells); the re-measurement uses a 60 s window (≈300 tail observations) at the same 50-connection operating point, 10 repeated runs. The 12-vs-60 s p99 rank correlation is $\rho = 1.00$ on PostgreSQL and $\rho = 1.00$ on MySQL; the corresponding p97.5-vs-p99 correlations are 0.98 and 1.00. The largest relative median shift is 5.6% and 3.8%, and the maximum run-level p99 CV is 7.3% and 3.2%, respectively. These quantities characterize within-campaign tail-window sensitivity rather than deployment-general uncertainty.

Layer	p99 12 s (ms)	p99 60 s (ms)	p97.5 60 s (ms)	p99 CV (%)	rps 60 s
PostgreSQL					
`pg-tuned`	18	18	17	3.2	3757.5
`pg`	18	19	16	7.3	3685.5
`knex`	27	26.5	24.5	5.7	2437.5
`drizzle`	34	34	32	3.7	1853
`typeorm`	49	48.5	46	1.8	1246
`prisma`	52	53.5	51.5	3.0	1161.5
`objection`	56	58	54.5	2.4	1029.5
`sequelize`	60	60.5	57.5	1.8	992.5
`mikroorm`	108	111.5	107	3.7	525
MySQL					
`mysql2-tuned`	25	25	24	2.9	2414
`mysql2`	28	28	26	2.6	2423
`knex`	30	30	28	2.3	2010.5
`drizzle`	48	48	45.5	2.3	1293.5
`typeorm`	57	56.5	55	3.2	1031
`sequelize`	68	67	63	1.8	892.5
`objection`	70	69	66	2.9	839.5
`prisma`	91	94.5	91	3.2	624.5
`mikroorm`	121	121	117	2.4	480.5

Table S24: Multi-worker deep-fetch throughput (req/s, median of three runs) as each layer is scaled to 1, 2, and 4 Express workers (node cluster, shared port). This bounded 1-to-4-worker sensitivity reports aggregate throughput for the displayed subset. It does not locate database-side saturation or establish scalability beyond four co-located application processes.

Layer	1 worker	2 workers	4 workers
PostgreSQL			
`pg`	3299	4055	3961
`prisma`	1117	2275	4449
`mikroorm`	435	898	1773
MySQL			
`mysql2`	2327	4698	8883
`prisma`	591	1167	2427
`mikroorm`	418	851	1624

23 Pool-size frontier on MySQL

Table S25 is the MySQL companion to the PostgreSQL pool sweep (Supplement Table S7): deep-fetch throughput as the connection pool varies from 1 to 50, per layer. Each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed pool of ten does not appear to drive the ranking on either engine.

Table S25: Deep-fetch throughput (req/s, MySQL, 50 connections, median of 3) as the connection pool varies from 1 to 50, per access layer. The primary campaign fixes the pool at 10; each layer saturates at a pool well below 50 and the ordering is preserved at every pool size, so the fixed-pool choice does not appear to drive the ranking.

Layer	1	2	5	10	20	50
`mysql2`	1545	2156	2352	2446	2506	2494
`knex`	1255	1926	1984	2081	2131	2120
`prisma`	464	572	631	639	626	648
`mikroorm`	416	469	486	418	492	429

24 Mixed read/write workload

Table S26 interleaves the deep-fetch read with single-row inserts at 90:10 and 70:30 read:write, both engines, with a physical write reset per run. The layer ordering matches the read ordering and is insensitive to the mix, because the deep-fetch read dominates the cost; exploratory.

Table S26: Mixed read/write workload: throughput (req/s) and p99 (ms) under an autocannon request mix interleaving the deep-fetch read with single-row inserts, at 90:10 and 70:30 read:write, both engines, median of five runs with an exact row-and-allocator reset per run. This is an exploratory sensitivity check; its observed ordering is interpreted from the displayed cells rather than assumed.

Engine	Layer	read:write	req/s	p99
PG	`pg`	90:10	4236	19
PG	`pg`	70:30	4347	18
PG	`knex`	90:10	3024	26
PG	`knex`	70:30	2993	26
PG	`prisma`	90:10	1504	55
PG	`prisma`	70:30	1409	58
PG	`mikroorm`	90:10	676	103
PG	`mikroorm`	70:30	705	99
MySQL	`mysql2`	90:10	2283	44
MySQL	`mysql2`	70:30	2505	47
MySQL	`knex`	90:10	2275	37
MySQL	`knex`	70:30	2232	40
MySQL	`prisma`	90:10	718	114
MySQL	`prisma`	70:30	729	103
MySQL	`mikroorm`	90:10	626	109
MySQL	`mikroorm`	70:30	639	105

25 Concurrency sweep

Figure S2 sweeps the deep fetch from 1 to 256 client connections for one layer of each tier, both engines. The 50-connection operating point used throughout sits above the knee for every layer on this pattern; the ordering is load-robust above about 32 connections. This is the evidence behind the choice of operating point.

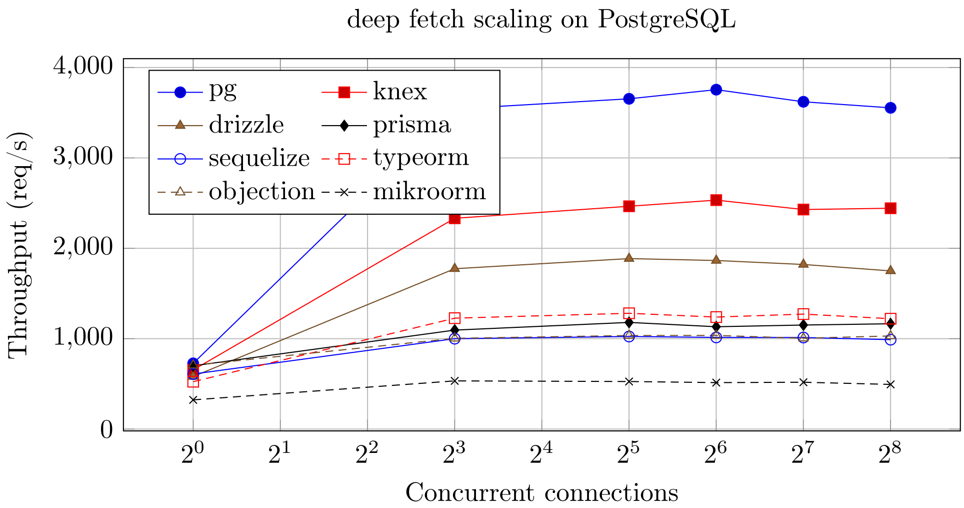

Figure S2: Deep-fetch throughput of each configured layer across the declared concurrency ladder (postgres). The curve maximum is used as an alternative empirical capacity denominator; uncertainty is assessed from the repeated point measurements.

26 Per-layer cross-backend-stack ranks

Table S27 reports PostgreSQL and MySQL ranks for all five patterns from the accepted full corrected-state campaign. The superseded mapping is preserved separately in `rq2-historical-provenance-comparison.json`; its ineligible MySQL cells are not treated as replication evidence. A separate clean same-host validation campaign remeasures the seven portable layers on the reviewer-prioritized deep-fetch, aggregation, and insert patterns. Between-campaign agreement is descriptive, not cross-host replication, and close reversals inside the predeclared ±5% remain exploratory.

27 Layer-by-backend-stack interaction magnitude

Table S28 reports the corrected-state PostgreSQL÷MySQL throughput ratio per layer and pattern with a within-campaign 95% paired-bootstrap interval.

Table S27: Corrected-state throughput ranks (1 = fastest) for the seven portable layers on PostgreSQL (PG) and MySQL. Every cell has 25 paired campaign repetitions, zero request failures, and a passing pre/post state check.

	Point read		Range		Deep fetch		Aggregation		Insert	
Layer	PG	MySQL	PG	MySQL	PG	MySQL	PG	MySQL	PG	MySQL
`knex`	1	1	1	1	1	1	4	2	1	3
`drizzle`	3	4	2	4	2	2	1	3	2	1
`prisma`	6	6	6	6	4	6	6	7	4	7
`sequelize`	5	5	5	5	6	4	5	5	5	4
`typeorm`	2	3	4	3	3	3	2	4	6	5
`objection`	4	2	3	2	5	5	7	6	3	2
`mikroorm`	7	7	7	7	7	7	3	1	7	6

It quantifies heterogeneity between the two tested stacks; because the DBMS, official adapter, lower-level driver, and protocol can change together, it is not a pure engine effect.

Table S28: Corrected-state PostgreSQL ÷ MySQL backend-stack throughput ratio (geometric mean of paired per-replicate ratios, with a within-campaign 95% paired-bootstrap interval) for the seven portable layers. The DBMS, official adapter, lower-level driver, and protocol can change together; ratios therefore do not identify a pure DBMS effect. The bottom rows report the randomized-block F statistic and permutation p value for heterogeneity among layer ratios within this campaign (20,000 permutations). A printed p of 0.0000500 is the resolution floor $1/(B+1) = 1/20{,}001$: the observed statistic exceeded every permutation, so the test bounds p from above and does not separate the patterns from one another.

Layer	Point read	Range scan	Deep fetch	Aggregation	Insert
`knex`	1.31 [1.30,1.32]	1.06 [1.05,1.07]	1.22 [1.21,1.24]	1.38 [1.35,1.41]	2.83 [2.69,2.99]
`drizzle`	1.44 [1.42,1.45]	1.28 [1.27,1.29]	1.41 [1.39,1.43]	1.60 [1.57,1.62]	2.40 [2.28,2.53]
`prisma`	1.59 [1.56,1.61]	1.58 [1.55,1.60]	1.84 [1.82,1.87]	1.82 [1.80,1.84]	3.06 [2.98,3.14]
`sequelize`	1.30 [1.27,1.32]	1.16 [1.14,1.17]	1.11 [1.10,1.12]	1.36 [1.34,1.39]	1.69 [1.62,1.77]
`typeorm`	1.39 [1.36,1.41]	1.16 [1.14,1.18]	1.20 [1.18,1.21]	1.53 [1.50,1.56]	1.96 [1.90,2.03]
`objection`	1.24 [1.22,1.26]	1.05 [1.03,1.07]	1.23 [1.21,1.24]	1.27 [1.25,1.30]	2.26 [2.15,2.37]
`mikroorm`	1.17 [1.15,1.20]	1.06 [1.04,1.08]	1.10 [1.07,1.13]	1.40 [1.38,1.42]	1.55 [1.49,1.60]
Blocked F	140.0	386.2	458.5	179.5	127.1
Permutation p	0.0000500	0.0000500	0.0000500	0.0000500	0.0000500

28 Study factors

Table S29 records which factors the benchmark holds constant, which it varies, and which are uncontrolled on the single virtualized host; the uncontrolled factors are why results are reported for this configuration rather than as a general library ranking.

Table S29: Factors in the benchmark: what is held constant, what varies (the treatments), and what is uncontrolled. The same-SQL contrast additionally holds the generated SQL and row mapping constant; because it changes query formulation, API, protocol, statement preparation, round-trip structure, and mapping *together*, its residual is a compound standardized contrast and does not isolate any single factor. The uncontrolled factors are inherent to a single virtualized host and are the reason results are reported for this configuration, not as general library performance.

Status	Factor	Detail
Held constant	schema, indexes, seed data	identical DDL + deterministic seed, both engines
	per-replicate request stream	paired PRNG stream, seeded on (endpoint, replicate)
	connection pool	fixed at ten for every layer
	response bytes	byte-identical JSON across layers (verified)
	host, OS, engine versions	one host; pinned July-2026 versions (Supplement Table S11)
	warm-up, run duration, cell order	15 s warm-up, 12 s runs, randomized order
Varies (treatments)	access layer	9 policy-selected documented + 2 tuned implementations
	backend stack	PostgreSQL or MySQL plus its supported adapter/driver
	access pattern	point read, range scan, deep fetch, aggregation, insert
Uncontrolled	hypervisor neighbours, CPU steal	single virtualized host
	thermal / frequency scaling	not pinned; bounded by the drift and post-restart checks
	database cache / kernel history	warm by design; randomized order + fresh boot mitigate

29 Paired significance of the p99 ladder

Table S30 is the p99 counterpart of the throughput significance table (Supplement Table S36): the same paired permutation, Wilcoxon, geometric-mean-ratio, paired-bootstrap-CI, and dominance analysis is applied to each adjacent layer's per-replicate deep-fetch p99 on both engines. Differences of about 1 ms are practically unresolved at the histogram resolution and are not promoted to substantive rank claims. These resolved gaps quantify high-load p99 under fixed demand. Together with the matched-utilization sensitivity they are consistent with a material capacity-and-queueing contribution, but do not identify a mechanism or establish different sub-saturation latency.

30 Extended methods (in the replication package)

The reporting checklist, the benchmarking-pitfalls checklist, the detailed statistical methods, and the measurement details are distributed with the replication package (`notes/supplement-methods.pdf`) rather than reproduced here, to keep this supplement compact; the numbered tables and figures above are unaffected.

Table S30: Paired comparison of adjacently ranked access layers on the deep fetch by response-time **p99** — the p99 counterpart to the throughput significance table (Supplement Table S36), over the 25 repeated runs on each engine. Layers are ranked by median p99 (lower is better) and each adjacent pair is compared on its per-replicate p99 ratios: median p99 (ms) with a within-campaign bootstrap 95% CI, the geometric-mean paired ratio B/A (how many times the slower-tail layer's p99 exceeds the faster's) with a paired bootstrap 95% CI, paired dominance (fraction of replicates in which B's p99 exceeds A's), and the two-sided paired sign-flip permutation p (20000 permutations; 5.0×10^{-5} is the resolution floor $1/(B+1)$, and the Wilcoxon signed-rank test agrees, replication package). This delivers the per-cell p99 inference the outcomes table (Supplement Table S34) lists as a primary analysis. These quantify high-load p99 differences under fixed demand; together with the matched-utilization sensitivity they are consistent with a material capacity-and-queueing contribution, but do not identify a mechanism or establish different sub-saturation latency. Like the throughput family, these pairs follow the observed ranking, so the supplement's multiplicity and exchangeability section states the Bonferroni conservatism check and the block assumption they rest on.

Comparison (A < B, p99)	med. A p99 [95% CI]	med. B p99 [95% CI]	ratio B/A [95% CI]	dom.	perm. p
PostgreSQL					
`pg` < `knex`	18 [17–20]	27 [25–28]	1.43 [1.36–1.52]	1.00	5.0×10^{-5}
`knex` < `drizzle`	27 [25–28]	34 [33–35]	1.27 [1.22–1.33]	0.98	5.0×10^{-5}
`drizzle` < `typeorm`	34 [33–35]	49 [48–50]	1.44 [1.39–1.49]	1.00	5.0×10^{-5}
`typeorm` < `prisma`	49 [48–50]	52 [51–53]	1.08 [1.06–1.10]	0.96	5.0×10^{-5}
`prisma` < `objection`	52 [51–53]	56 [54–58]	1.08 [1.06–1.11]	0.92	5.0×10^{-5}
`objection` < `sequelize`	56 [54–58]	60 [60–61]	1.07 [1.05–1.10]	0.92	5.0×10^{-5}
`sequelize` < `mikroorm`	60 [60–61]	108 [107–112]	1.81 [1.77–1.85]	1.00	5.0×10^{-5}
MySQL					
`mysql2` < `knex`	28 [27–28]	30 [29–31]	1.09 [1.07–1.12]	0.96	5.0×10^{-5}
`knex` < `drizzle`	30 [29–31]	48 [46–49]	1.58 [1.55–1.61]	1.00	5.0×10^{-5}
`drizzle` < `typeorm`	48 [46–49]	57 [56–58]	1.19 [1.17–1.22]	1.00	5.0×10^{-5}
`typeorm` < `sequelize`	57 [56–58]	68 [67–68]	1.19 [1.17–1.21]	1.00	5.0×10^{-5}
`sequelize` < `objection`	68 [67–68]	70 [68–70]	1.02 [1.01–1.04]	0.68	0.0223
`objection` < `prisma`	70 [68–70]	91 [89–94]	1.32 [1.29–1.35]	1.00	5.0×10^{-5}
`prisma` < `mikroorm`	91 [89–94]	121 [118–126]	1.34 [1.31–1.37]	1.00	5.0×10^{-5}

31 Artifact reproducibility summary

Table S31 summarizes what is needed to reproduce the study and which results are regenerated automatically; `REPRODUCE.md` in the replication package is the single runnable entry point.

Table S31: Artifact reproducibility summary: version and identifier, requirements, how to run, time and resources, and which results are reproduced automatically.

Item	Detail
Artifact & version	Complete revision artifact v1.13.17: seed-specification oracle, corrected-state 90-cell primary campaign, 42-cell same-host validation campaign, fresh secondary measurements, external-audit dataset, and blind review packets.
Code provenance	Git tag `v1.13.17` identifies the exact release commit, also recorded in the GitHub and Zenodo metadata. The package-lock SHA-256 and aggregate/per-file SHA-256 cover 42 measurement, admission, setup, state-control, schema, and adapter files. Exact 42-file source archives are preserved for both accepted campaigns and verify member-by-member against their embedded manifests. The accepted primary aggregate is `532d9c7ffe94...d06cf228`; the independently ordered same-host validation aggregate is `d32a32b5b68a...134d3789a`. Full values are retained in the source manifests.
Persistent identifier	Concept DOI `10.5281/zenodo.21313858`, resolving to release v1.13.17; the Zenodo record also carries an immutable per-version DOI.
Software	Node.js 24.18.0, PostgreSQL 18.4, MySQL 9.7.1, `npm`. Reference path: conda user-space (`conda-forge`); the Docker path pins the same versions by digest but starts relaxed durability in a containerized topology, so it reproduces the workflow rather than the headline default-durability insert numbers.
Hardware & disk	Single Linux host (full host and version capture in Table S11); ≈1 GB for the seeded databases.
Deterministic seed	2,000 authors / 100,000 posts / 1,000,000 comments from a fixed PRNG.
Smoke test (≈5–10 min)	`npm ci`; `npm run migrate && npm run seed`; `npm run bench:quick`; `npm run verify:spec` (seed-derived expected results), `npm run verify:seed-parity` (cross-engine fixture values), `npm run verify:fixed` (differential equivalence), `npm run verify:campaign-state`, and `npm run verify:writes` (write-state correctness, must print ALL WRITES SEMANTICALLY CORRECT).

Full campaign	Primary matrix: `npm run campaign:rq2` produces 90 cells with 25 runs each, followed by `npm run analyze:rq2`. The reduced same-host validation uses `npm run campaign:rq2-validation` and produces 42 cells with 25 runs each. Both commands fail closed on request errors or an incomplete roster and record pre-warm-up startup retries separately. Secondary experiments require additional hours and are invoked individually in `REPRODUCE.md`.
Regenerate from raw	≈minutes, *no database*: the standalone no-database renderers rebuild their mapped tables and figures from `results/*.json`, then `npm run sync:tables` and `make`; `MANIFEST.md` explicitly marks authored tables, the one pre-generated statement-log table, and seven run-coupled renderers.
Verification status	On 1 August 2026, an author-run isolated reconstruction from a clean export of the tagged commit verified 67/67 archived JSON hashes. Of the 58 committed tables, 48 carry a generator and 10 are authored analytical items with none; the documented no-database chain regenerated 42 byte-for-byte, while six generator-backed tables (S2, S6, S14, S19, S25 and Figure S2) were byte-compared unchanged because their renderer is run-coupled or their input, server statement logs, is not archived, and the 10 authored items were byte-compared unchanged (`notes/reconstruction-2026-08-01.md`). The earlier 26 July run covered the 60 datasets listed at that date; seven were added to the manifest afterwards. The earlier author-run archive-isolated v1.12.9 check verified 35/35 inputs and 45/50 then-committed tables; its five presentation-only differences are named in `notes/clean-room-reproduction.md`. Neither check is independent reproduction or a re-execution of the current measurements.
Entry point	`REPRODUCE.md` (smoke test, full matrix, archive-isolated author reconstruction from the tarball); the per-table data-and-generator map is in `MANIFEST.md`.

Table S32: Scope of every experiment: the access pattern(s), engine(s), number of access-layer implementations, and repeated runs each covers. The *primary comparison* (throughput and closed-loop p99) is the full five-pattern, two-engine matrix; every other experiment is a limited condition, most restricted to the deep fetch, and each conclusion is scoped accordingly. "Layers" counts implementations (the nine access layers plus, where present, the two tuned native baselines); a selected subset is used where a caption says so. Engine entries name where the experiment was run (`pg` is PostgreSQL-only, `mysql2` MySQL-only).

Experiment	Pattern(s)	Engines	Layers	Runs
Primary matrix: throughput & closed-loop p99 (main paper)	all five	PG, MySQL	11	25 independent
Same-SQL raw-path contrast (Table S14)	deep fetch	PG, MySQL	9	median of 10
Capacity-denominator sensitivity (Table S45)	deep fetch	PG, MySQL	8	archived-data re-expression
Open-loop CO-corrected tail (Tables S6, S17)	deep fetch	PG, MySQL	5	rate sweep
Utilization-controlled tail (Tables S21, S22)	deep fetch	PG, MySQL	8	median of 5
Equal-CPU, 1/2/4 cores (Table S4), exploratory	deep fetch	PG	3	median of 3
Node-cluster multi-worker (Table S24)	deep fetch	PG, MySQL	3	median of 3
Pool-size sweep, 1–50 (Tables S7, S25)	deep fetch	PG, MySQL	4	median of 3
Mixed read/write, 90:10 & 70:30 (Table S26)	deep fetch, insert	PG, MySQL	5	median of 5
Relaxed vs default durability (Table S3)	insert	PG, MySQL	11	25 default plus 5 relaxed
Wait-event flush decomposition (Table S19)	insert	MySQL	3	4000 inserts/layer
Transactional multi-statement write (Table S8)	`POST /threads`	PG	5	median of 5
Longer-window (60 s) tail (Table S23)	deep fetch	PG, MySQL	11	10 runs
Canonical-constructor microbenchmark (Table S44)	all read shapes	none	shared code	30 blocks
Post-restart cold cache (Table S20)	deep fetch	PG, MySQL	3	vs primary median
Alternative eager-loading (Table S18)	deep fetch	PG, MySQL	3	median of 5
Same-host validation campaign	deep fetch, aggregation, insert	PG, MySQL	7 portable	25 independent

Specification-derived read oracle (unnumbered panel in the semantic-admission section)	all read methods	PG, MySQL	9 per engine	1,000 random/type + boundary/list + six fan-out fixtures
External protocol audit (Table S37)	9 source benchmarks	n/a	7 stages	1 author-rater

32 Construct validity of the policy-selected documented treatment

The *policy-selected documented* rule is a mechanical, predeclared treatment-selection policy with no behavioral practitioner interpretation. It selects the first relations API on the version-compatible official page under the stated conflict rule. For auditability, the artifact preserves the exact response returned by the nearest available Wayback capture at or before the freeze for each official URL; the versioned documentation-snapshot manifest (`manifest.json`) records the committed HTML, capture timestamp, SHA-256, pinned version, selected API, alternatives, tie-break status, and decision basis; the same per-layer assignment appears in `METHODOLOGY.md` and Table S16. A capture establishes page state only at its timestamp, not continuous lack of change through the freeze. This provides author-replay provenance, not evidence of production frequency, optimality, or independent agreement. The author applied every assignment; two result-blind selection reviews are prepared but pending. Table S33 records this per layer. Drizzle is the one borderline case: it exposes both a SQL-style query builder (the selected join) and a higher-level relational-query API that its documentation also presents, so we selected the builder join under that base-layer rule — the page presents it as the core SQL-style layer — and record the relational-query API as the documented alternative. Where the documented alternative is a byte-identical drop-in, the loading-strategy sensitivity check (Table S18) adds an empirical signal: for the join-selected ORMs (Sequelize, MikroORM) the policy-selected documented path is the *faster* of the two strategies, so a layer's deep-fetch deficit does not appear to be an artifact of a poor selected strategy; for Objection the documented default (batched select-in) is the slower option on MySQL, which we disclose rather than correct; the Drizzle, Prisma, and TypeORM alternatives were not evaluated (TypeORM's errors on the ordered deep fetch), itself a portability caveat. The construct is therefore a reproducible policy-defined treatment, not a practitioner persona and not a claim about performance-tuned expert usage or measured production frequency; RQ1 is scoped accordingly (main text, Threats to Validity).

Table S33: Construct-validity record for the policy-selected documented deep-fetch treatment: the registered API, what the frozen page establishes or leaves ambiguous, and the empirical loading-strategy signal where the documented alternative is a byte-identical drop-in (Table S18).

Layer	Selected API	Frozen-page evidence / ambiguity	Loading-strategy signal
`pg`, `mysql2` (+tuned)	hand-written JOIN	No relation API; parameterized driver query path	n/a (hand-written SQL)
`knex`	builder `.join()`	Builder join page; no relation abstraction	n/a (hand-written SQL)
`drizzle`	builder join	Two co-prominent paths; documented-base-layer tie-break selected builder	alternative (relational-query API) not evaluated
`prisma`	`include`	Relations page presents `include`; alternative recorded	alternative (join strategy) not evaluated
`sequelize`	`include` (join)	Eager-loading page presents `include`	selected path *faster* than select-in
`typeorm`	`relations`	Find-options page presents `relations`	alternative errors on the ordered deep fetch (not evaluated)
`objection`	`withGraphFetched`	Eager-methods page presents `withGraphFetched`	selected path (select-in) slower than join on MySQL (disclosed)
`mikroorm`	`populate` (join)	Populate page presents `populate`	selected path *faster* than select-in

33 Results roadmap

Table S34 classifies every result by analysis role: primary measurements, secondary controls, or exploratory sensitivity evidence. It is the navigational companion to the reproduction map in `experiments/MANIFEST.md`, which ties each table to its generator and input data. The policy-selected documented configuration is the RQ1 estimand; common-SQL, alternative-loading, and operating-point experiments are sensitivity analyses, not alternative practitioner personas.

34 Per-pattern capacity and utilization at the operating point

The operating-point protocol requires that capacity be identified so the fixed operating point can be read against it (main text, Study Design). The deep-fetch concurrency sweep (Section 25, Figure S2) does this in per-layer detail for one pattern; review point 6.5 asks whether the other four patterns, reported only at the fixed 50-connection demand, sit at comparable utilization. To answer it we swept every access layer over a connection ladder (1, 4, 8, 16, 32, 50, 100, 200) on *all five* patterns and both engines, medianed over three runs per point after a warm-up, and for each layer computed its esti-

Table S34: Outcomes and their analysis role. The primary outcomes carry the paired inference and the research-question claims; the secondary and exploratory outcomes are reported descriptively as controls and sensitivity analyses that constrain or contextualize the primary results, and are not treated as additional confirmatory hypotheses. RQ2 is answered from the primary throughput outcome: the layer×backend-stack interaction, the rank correlations, and the reversal screen are secondary *analyses of* that primary measurement, not a separate confirmatory outcome, which is why no RQ2 quantity is promoted to primary.

Role	Outcome	Analysis
Primary	Throughput (req/s), 5 patterns × 2 engines, policy-selected documented layers, 50-connection operating point	Bootstrap median CI; spread and rank. Adjacent-pair paired permutation and Wilcoxon are reported for the deep fetch only, and are post hoc
Primary	Closed-loop p99 tail latency, same cells	Per-cell bootstrap CI on p99; adjacent-pair paired tests on the deep fetch only, post hoc
Secondary	Same-SQL (raw-path) contrast	Describes the residual (compound) same-SQL difference
Secondary	Sub-saturation open-loop tail (coordinated-omission corrected)	Lower-load latency companion to the high-load primary p99
Secondary	Layer×backend-stack interaction (per pattern)	PG÷MySQL stack ratios (S28), blocked permutation (F on D)
Secondary	Combined app+db CPU efficiency, equal-CPU control	End-to-end compute view; operational confirmation of the CPU trade
Explor.	Aggregation fan-out; OFFSET vs. keyset; pool-size sweep; durability & isolation sensitivity; RSS	Descriptive; pitfall illustration and robustness checks

mated saturating throughput (the ladder maximum), the utilization at 50 connections (throughput at 50 connections divided by that maximum), and the knee (the smallest connection count reaching 95% of the maximum).

Table S35 reports the result. The knee lies at or below 50 connections for every pattern on both engines, so the fixed 50-connection operating point places all five patterns at high utilization of their own capacity: cross-pattern comparisons of p99 and spread at that point are therefore made at comparable (high) relative utilization, not at unknown or widely different fractions of capacity. The binding constraint is the ten-connection pool shared by every pattern (main text, Controls), which is why the knee clusters well below 50 connections regardless of the pattern's absolute throughput. This extends the capacity-identification stage of the protocol from the deep fetch to all five patterns; the equal-utilization open-loop tail (Section 20) and the exploratory equal-compute check (Section 4), which each need a per-layer saturating-throughput *target*, remain demonstrated on the deep fetch, the pattern on which the layer spread is largest.

Table S35: Per-pattern capacity and utilization at the fixed 50-connection operating point, from a concurrency sweep (connection ladder 1–200) of every access layer on all five patterns and both engines. *Knee* is the median (across layers) smallest connection count reaching 95% of a layer's own estimated saturating throughput; *util. @50* is the median (and, in parentheses, minimum) across layers of the ratio (throughput at 50 connections)/(estimated saturating throughput). The reported values show where the fixed 50-connection point lies relative to each observed sweep maximum. Capacity characterization is reported for all five patterns; the equal-utilization and equal-compute conditions are evaluated separately on the deep fetch.

	PostgreSQL		MySQL	
Pattern	knee	util. @50 (min)	knee	util. @50 (min)
Point read	32	99% (96%)	16	100% (92%)
Range scan	16	100% (97%)	16	99% (96%)
Deep fetch	16	98% (92%)	8	99% (91%)
Aggregation	32	99% (88%)	32	100% (95%)
Insert	32	99% (87%)	16	99% (97%)

35 Paired significance of the deep-fetch ranking

The main text foregrounds paired *effect sizes* for the deep-fetch ranking — geometric-mean per-replicate throughput ratios and the fraction of replicates in which the faster layer wins — because they, not the significance tests, carry the ordering. Table S36 reports the post-hoc adjacent-pair tests as a descriptive companion: since the ranking itself selects which pairs are compared, the inference is conditional, so the permutation and Wilcoxon results are not treated as confirmatory. The bootstrap intervals are within-campaign (main text, Study Design), not general over deployment environments.

36 Application-tier CPU trade-off

Figure S3 plots each layer's application-tier CPU against its throughput on the deep fetch, the resource-accounting stage the main text lists among its operating conditions: every layer draws about one application core, and the native driver leads on throughput and on requests per CPU-second (application-tier and combined), consistent with its leaner SQL; these CPU shares, measured at unequal throughput, do not identify where per-request work occurs.

Table S36: Paired comparison of adjacently ranked access layers on the deep fetch (MySQL), over the 25 repeated runs. Because the design is a randomized block — every layer measured once per replicate, from the same seeded identifier distribution — layers are compared on their per-replicate throughput ratios: median throughput (req/s; the per-cell bootstrap CI is in the pattern tables), the geometric-mean paired ratio A/B with a within-campaign paired bootstrap 95% CI, paired dominance (fraction of replicates in which A exceeds B), Cliff's delta δ on the unpaired samples, and the two-sided paired sign-flip permutation p (20000 permutations; a value of 5.0×10^{-5} is the resolution floor $1/(B+1) = 1/20{,}001$, i.e. the observed statistic exceeded every permutation; the Wilcoxon signed-rank test agrees, replication package). These pairs follow the observed ranking rather than a preregistered one, so they are a secondary descriptive check; the supplement's multiplicity and exchangeability section states the Bonferroni conservatism check and the block assumption the paired tests rest on.

Comparison (A > B)	med. A	med. B	ratio A/B [95% CI]	dom.	δ	perm. p
mysql2 > knex	2416	1982	1.22 [1.21–1.24]	1.00	1.00	5.0×10^{-5}
knex > drizzle	1982	1301	1.53 [1.51–1.55]	1.00	1.00	5.0×10^{-5}
drizzle > typeorm	1301	1017	1.27 [1.25–1.29]	1.00	1.00	5.0×10^{-5}
typeorm > sequelize	1017	880	1.16 [1.15–1.17]	1.00	1.00	5.0×10^{-5}
sequelize > objection	880	836	1.06 [1.04–1.07]	0.98	0.95	5.0×10^{-5}
objection > prisma	836	630	1.32 [1.30–1.33]	1.00	1.00	5.0×10^{-5}
prisma > mikroorm	630	472	1.35 [1.33–1.38]	1.00	1.00	5.0×10^{-5}

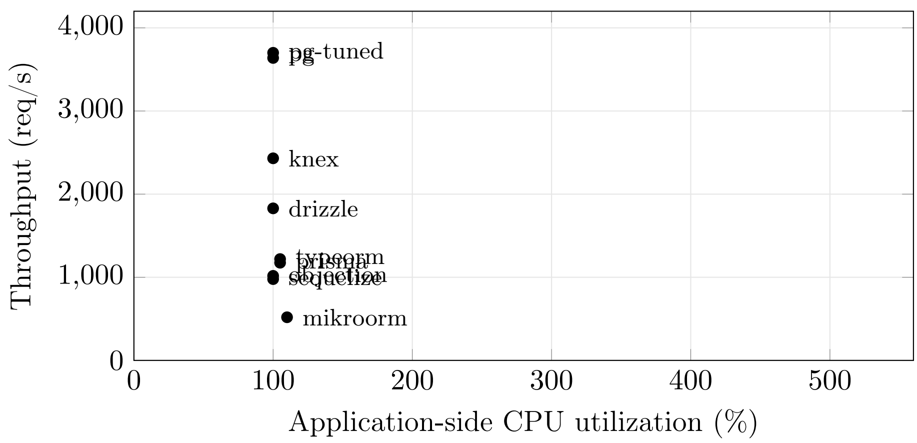

Figure S3: Throughput versus application-side CPU on the deep fetch (PostgreSQL, ten-connection pool, medians of 25 replicates). Every layer sits at about 100–110% of one application core; the native driver leads on the throughput axis, and the slower ORMs are low on throughput at the same CPU. CPU is the server process only; database-side CPU is excluded for all layers alike.

37 Retrospective application of the protocol to prior benchmarks

Table S37 reports a structured audit of the nine benchmark sources retained by the scoping search. The codebook fixes five judgement codes and seven stage definitions. The machine-readable audit preserves all 63 judgements with source locators and paraphrased evidence, and its validator checks completeness. No benchmark was rerun, and *not reported* is not treated as proof that a procedure never occurred. The author performed the coding, so the exercise demonstrates a bounded application of the protocol and the narrower conclusions it produces, not inter-rater reproducibility or independent validation. Result-blind forms for additional raters are shipped in the artifact reviewer packet.

38 Specification-derived read admission and differential equivalence

The mandatory read gate has two layers. First, `verify-spec.mjs` imports an executable seed specification that replays the fixed data PRNG and campaign fan-out fixture without querying either database or importing any adapter. It derives expected scalar values, field sets, ordering, graph membership, and author aggregates for 1,000 random post and author identifiers, explicit boundary and list cases, and all six fan-out fixtures. Database-generated timestamps are checked for presence and canonical ISO-8601 representation. The specification-derived panel below reports 73,080 passing expected-result checks. A separate cross-engine fixture gate compares every declared fan-out value and generated identifier, excluding only DBMS-generated timestamps; its 662-row representation is identical on PostgreSQL and MySQL (SHA-256 `402e0aebbbc9...a99fe5ca`). The full hash is retained in the artifact. This supplies ground truth relative to the declared deterministic task for the tested inputs; it is finite evidence, not proof for arbitrary programs.

Specification-derived read admission. Expected fields, values, ordering, graph membership, and aggregates are computed by replaying the deterministic seed, not from native-driver responses. Timestamps are checked for presence and canonical ISO-8601 representation because their instants are database-generated.

Engine	Adapters	Post inputs	Author inputs	List cases	Checks	Failed
PostgreSQL	9	1015	1007	8	36,540	0
MySQL	9	1015	1007	8	36,540	0

Second, `verify.mjs` checks 12 fixed probes and `verify-property.mjs` differentially compares the implementations over a separate fixed-seed random and edge-case sweep. Table S38 reports 60,800 adapter-versus-native

Table S37: Single-rater retrospective protocol audit of the nine benchmark sources retained by the scoping search. Y = reported as satisfying the codebook (no source attained Y); P = partial; – = not reported; n/a = outside the access-layer estimand. No source was rerun. The author performed the coding, so this demonstrates a bounded application of the protocol but not inter-rater reproducibility. Per-cell source locators and evidence are preserved in `external-protocol-audit.json`.

Source	**M1**	**M2**	**R3**	**R4**	**R5**	**R6**	**R7**	**Conclusion licensed by reported controls**
Colley et al. (2018)	–	P	P	–	–	P	–	Describes the selected ORM/non-ORM query examples; it does not establish a general ordering among ORM products or under load.
Yusmita et al. (2025)	–	P	P	–	–	P	–	Describes the two authored strategies; it does not isolate intrinsic ORM overhead or behavior at located capacity.
Attala and Khemapatpapan (2025)	–	P	–	–	–	P	–	Supports comparison of the implemented configurations on its workloads, not a library-only or saturation-relative ordering.
Zhadko-Bazilevych (2025)	–	P	–	P	–	–	–	Supports an exploratory ordering of the implemented modes; the parallel-load result is not a matched-capacity client-tail comparison.
Pratama and Raharja (2023)	–	P	P	–	P	–	–	Reports the named framework–library–environment cells; simultaneous variation and unspecified task equivalence prevent an isolated access-layer ordering.
Salunke and Ouda (2024)	n/a	n/a	n/a	P	n/a	P	n/a	Its engine estimand is outside the access-layer protocol; no access-layer conclusion should be inferred.
Drizzle benchmark suite	–	P	–	P	P	P	P	Describes public target implementations at the dashboard load; it does not support utilization-matched or library-intrinsic attribution.
Prisma query-performance benchmarks	P	P	–	–	–	–	P	Describes median latency for the visible query implementations; it does not establish throughput, tail, or capacity behavior.
IMDBench / Gel Data	P	P	P	–	P	–	P	Describes the public implementations and workload point; it does not separate strategy from layer or generalize across utilization.

M1 semantic admission; M2 treatment definition; R3 common-SQL raw-path sensitivity; R4 capacity characterization; R5 operating-point separation; R6 resource accounting; R7 implementation-waste evidence. *Not reported* is not evidence that a control was never performed.

comparisons with no divergence. Here the native result is an equivalence comparator, not the expected-result oracle. A common error could pass this layer but fail the seed-derived layer; conversely, mutual disagreement identifies a comparability failure even when one implementation matches a small expected sample.

Table S38: Randomized differential semantic-equivalence coverage (`bench/verify-property.mjs`). A fixed-seed generator (mulberry32, seed 99005011) draws inputs across the full key range; the native response is the differential comparator and every adapter response must be byte-identical to it, with any thrown error captured as a comparable value so an input-dependent crash also counts as a divergence. The gate is re-run against the seeded database (2,000 authors, 100,000 posts, 1,000,000 comments), not derived from `results/raw.json`.

Dimension	Coverage
Read methods checked	`getPost`; deep fetch (`getThread`); same-SQL deep fetch (`getThreadRaw`); aggregation (`authorSummary`); keyset range scan (`listPosts`)
Seeded-random inputs	1,000 post IDs + 1,000 author IDs, uniform over the full range
Edge cases	9 post IDs (0, −1, 1, 2, 99,999, 100,000, 100,001, 101,000, $2^{31}-1$); 7 author IDs (0, −1, 1, 1,999, 2,000, 2,001, 3,000); 8 keyset pages (empty / boundary; limit 0, 1, 100)
Distinct inputs / adapter	3,800 (after de-duplication)
Adapters vs. baseline	8 per engine
Comparisons	30,400 per engine; 60,800 across PostgreSQL and MySQL
Divergences	**0** (ALL BYTE-IDENTICAL)

39 The five access patterns

Table S39 lists the five HTTP endpoints of the benchmark workload, the access pattern each realizes, and what each is intended to stress. The main text (Study Design) describes the patterns in prose; this table is the compact reference.

Table S39: The five access patterns and what each stresses.

Endpoint	Pattern	Stresses
`GET /posts/:id`	point read (PK)	per-query overhead
`GET /posts?before=`	keyset range scan	index seek + hydration
`GET /posts/:id/thread`	deep fetch	join strategy, N+1 avoidance
`GET /authors/:id/summary`	aggregation	grouped counts / raw SQL
`POST /posts`	insert	write path

40 Protocol compliance levels and their coverage

The main text (Study Design) defines the six protocol compliance levels and their mapping to the seven protocol stages: M1 and M2 form the mandatory validity core, and R3–R7 supply five claim-specific extensions. Table S40 records precisely which workload cells satisfy each in this experiment. The mandatory validity core and capacity characterization hold across all five patterns on both engines; the three extensions — common-SQL sensitivity, utilization-controlled latency, and resource accounting — are demonstrated on the deep fetch, the widest-spread tested pattern, as a representative-point instantiation rather than a claim of full-matrix coverage. The sixth, implementation-waste level is satisfied mechanically for the measured source by the static dead-work check; its stronger independent-human-review form is not: packets exist under `notes/reviewer-packets/`, but no external human review is claimed. Stating the levels and their coverage explicitly is what makes the protocol reusable: a later study can report its own compliance level per cell against the same ladder.

41 Predefined contrasts against the native-driver reference

Selecting adjacent pairs *after* seeing the ranking makes those tests conditional on the observed ordering, so their p-values add little (main text, Measurement stability; the adjacent-pair table is Table S36). A predeclared contrast set avoids this. Table S41 fixes the reference (the native driver) and the contrasts (each portable layer against it) in advance, independently of the ranking, and reports for each layer on the deep fetch the geometric-mean paired ratio of native-to-layer throughput, a within-campaign 95% paired-bootstrap interval, and the paired dominance. Every portable layer trails the native driver in all 25 replicates on both engines (dominance 1.00), and the ratios range from 1.49× (`knex`, PostgreSQL) to 7.04× (`mikroorm`, PostgreSQL) — essentially the same spread (7.04 against the main text's 7.01, a different estimator), now as predeclared contrasts rather than post-hoc pairwise tests. The contrasts are within-campaign, not deployment-general.

42 Run-to-run p99 spread

Each cell's reported p99 is the median of 25 run-level p99 values, and each run-level p99 is itself estimated from roughly the top 1% of that run's requests, so a single run's p99 is noisy. Figure S4 plots all 25 per-run p99 values behind each deep-fetch cell (PostgreSQL) with the median marked, so the run-to-run stability of the estimate is visible rather than summarized by a single number: the spread is tight for the fast layers and wider for the

Table S40: Protocol compliance levels and which workload cells satisfy each in this experiment. Each level names the protocol stage (main text, Figure 1) it requires: the six levels group the seven stages by the claim each licenses, with M1 and M2 combined into the mandatory validity core. The mandatory validity core and capacity characterization are exercised across the *whole* matrix; the common-SQL, utilization, and resource extensions are demonstrated on the deep fetch — the widest-spread tested pattern (main text, Results) — as a representative-point instantiation, not a claim of full-matrix coverage. This is what the protocol box means by recommended stages that "may be scoped to a representative workload point."

Compliance level (protocol stages)	Requirement it adds	Cells satisfying it here	Evidence
Mandatory validity core (M1, M2)	Specification-derived expected results, differential equivalence, correct write state, and a predeclared treatment rule, on *every* timed cell	All five patterns × two engines (all 90 cells)	Study Design; Table S38; write verifier
Capacity characterization (R4)	Locate estimated saturating throughput by a concurrency sweep at each workload point	All five patterns × two engines	Table S35
Common-SQL-sensitivity extension (R3)	Add a common-SQL raw-path sensitivity contrast	Deep fetch × two engines	Tables S14, S42
Utilization-controlled extension (R5)	Measure the tail at matched fractions of each treatment's own capacity	Deep fetch × two engines	Tables S21–S22, S43
Resource-accounting extension (R6)	Report compute/memory demand or add an equal-budget sensitivity	Deep fetch on both engines; equal-CPU sensitivity is a PostgreSQL subset	Tables S4, S5, S10
Implementation-waste extension (R7)	Recorded evidence that no treatment computes or fetches values it then discards on the timed path; independent human review is a separate, stronger level	Static dead-work check clean on all 13 modules of the measured source (11 adapters plus the two shared canonical helpers); the runtime diagnostic gate is installed for subsequent campaigns and did not gate this one	`results/dead-work.json`; `bench/dead-work.test.mjs`

Table S41: Predefined contrasts against the native-driver reference on the deep fetch, fixed in advance rather than selected after the ranking (reviewer 8.1). For each portable layer: the geometric-mean paired ratio of *native-driver to layer* throughput (how many times faster the native driver is, per replicate), a within-campaign 95% paired-bootstrap interval, and the paired *dominance* (fraction of the 25 replicates in which the native driver is faster). Because the reference and the contrast set are predeclared, these are not conditional on the observed ordering, unlike the adjacent-pair tests. Values are within-campaign, not deployment-general; the reference is `pg` on PostgreSQL and `mysql2` on MySQL. These paired geometric means differ slightly from the main text's spreads (7.04 against 7.01 on PostgreSQL, 5.19 against 5.12 on MySQL) because that figure is a ratio of cell medians, a different estimator of the same contrast rather than a disagreement.

	PostgreSQL		MySQL	
Layer	native/layer [95% CI]	dom.	native/layer [95% CI]	dom.
`knex`	1.49 [1.47,1.51]	1.00	1.22 [1.21,1.24]	1.00
`drizzle`	1.98 [1.95,2.01]	1.00	1.87 [1.85,1.90]	1.00
`prisma`	3.10 [3.06,3.15]	1.00	3.84 [3.81,3.87]	1.00
`sequelize`	3.70 [3.65,3.74]	1.00	2.76 [2.74,2.78]	1.00
`typeorm`	2.97 [2.94,3.00]	1.00	2.38 [2.36,2.40]	1.00
`objection`	3.55 [3.50,3.60]	1.00	2.92 [2.88,2.95]	1.00
`mikroorm`	7.04 [6.90,7.18]	1.00	5.19 [5.12,5.27]	1.00

slow ones, consistent with the coarser p99 estimate at higher latency, and the medians preserve the reported ordering. The load generator reports latency percentiles rather than raw per-request latencies, so a full HDR-histogram reconstruction is left to future work; the median-of-run-level-p99 estimator, its within-campaign bootstrap interval, and the longer-window (60 s) sensitivity check (Section 21) are the mitigations used here.

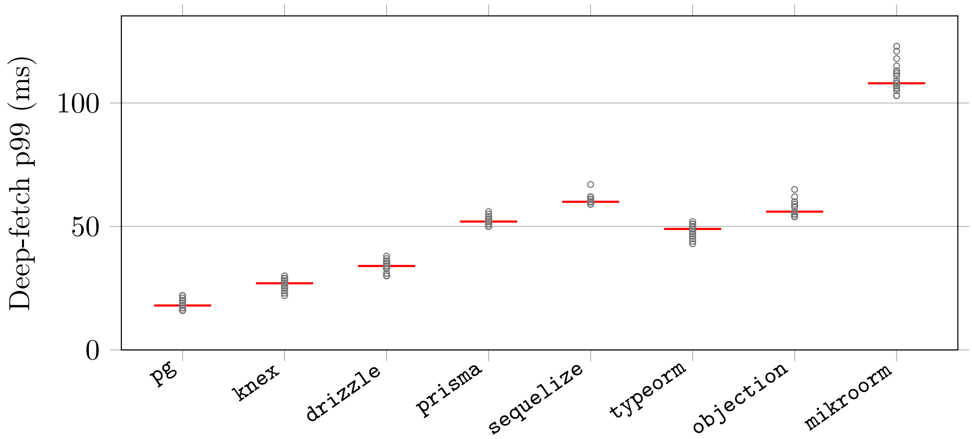

Figure S4: The 25 per-run p99 values behind each reported deep-fetch p99 on PostgreSQL (red bar = median, the value tabulated in the main text). Each run's p99 is estimated from roughly the top 1% of that run's requests, so a single run's p99 is noisy; the paper reports the *median* of these 25 run-level p99 values with a within-campaign bootstrap interval rather than pooling dependent requests. The plot shows the run-to-run dispersion this median summarizes: it is tight for the fast layers and widens for the slower ones, consistent with the coarser p99 estimate at higher latency. A full per-request HDR recorder was not used (the load generator reports latency percentiles, not raw request latencies), so an HDR reconstruction is left to future work.

43 Components moved by the common-SQL raw-path sensitivity contrast

The common-SQL raw-path sensitivity contrast (main text, RQ1) changes more than SQL text. Replacing each layer's policy-selected relation-loading path with the same hand-written SQL through its raw facility changes several mechanisms at once. Table S42 enumerates them. The residual spread it leaves is a common-SQL raw-path contrast — not a bound on the combined effect, and not attributable to any single component, in particular not to SQL formulation or query count.

Table S42: The *common-SQL raw-path sensitivity contrast* is a compound intervention: switching each layer from its policy-selected documented path to the same hand-written SQL through its raw facility moves every component below at once — equalizing the SQL text, query strategy, round-trip count, and result mapping, while the raw-SQL facility, statement preparation, and disclosed wire protocol still differ across layers. The residual difference it leaves is a common-SQL raw-path contrast, not a bound on their combined effect, and cannot be attributed to any single component — in particular, not to SQL formulation or query count.

Component	Policy-selected documented path	Same-SQL (raw-path) contrast
Loading API	the ORM's eager-load API (`include`, `populate`, ...) or a builder join	the layer's raw-SQL facility
Query strategy	per layer: single join, batched select-in, or multi-query	one hand-written two-statement join, identical across layers
SQL formulation	per-ORM generated statement text	byte-identical statement text (server-side confirmed)
Round-trip count	one to four statements	two statements
Statement preparation	ORM-managed	the raw facility's default
Wire protocol	per driver (extended/simple, text/binary)	per driver, unchanged and disclosed
Result hydration	the ORM's object-graph mapping	shared canonical constructors, identical across layers

44 Deep-fetch tail: equal closed-loop demand and matched utilization

The high-load p99 under equal closed-loop demand (main text, Tail latency) and the p99 at matched fractions of each layer's own capacity read the deep-fetch tail two ways. Table S43 shows both per layer. At equal demand, p99 spans 18–108 ms on PostgreSQL and 28–121 ms on MySQL; at the stable nominal 50% target it contracts to 2.3–4.9 ms and 1.9–5.5 ms, respectively. Most 70% cells also remain small. This is consistent with a material capacity-and-queueing contribution but does not identify a causal share or a deployment-general latency bound. The unstable 85–95% cells are excluded from the contraction claim.

Table S43: The deep-fetch tail read two ways, per layer and engine. *Equal demand* is the high-load response-time p99 (ms) at the fixed 50-connection closed-loop point — the headline deep-fetch p99, which at this saturating load largely tracks capacity and connection-acquisition queueing. *50% util.* is the coordinated-omission-corrected p99 when each layer is instead offered half of its $\widehat{C}_{50}$ capacity proxy (Supplement Tables S21–S22; denominator sensitivity, Table S45). In this campaign the equal-demand and 50%-utilization ranges are 18–108 and 2.3–4.9 ms on PostgreSQL, and 28–121 and 1.9–5.5 ms on MySQL. The contraction at this stable sub-saturation target is consistent with capacity and connection-acquisition queueing contributing materially to the high-load gap; it does not establish a deployment-general latency bound.

		PostgreSQL p99 (ms)		MySQL p99 (ms)	
Layer	Category	equal demand	50% util.	equal demand	50% util.
`pg`	native-driver	18	2.6	–	–
`mysql2`	native-driver	–	–	28	4.7
`knex`	query-builder	27	2.3	30	1.9
`drizzle`	orm	34	2.5	48	3.9
`prisma`	orm	52	4.9	91	5.5
`sequelize`	orm	60	2.8	68	3.1
`typeorm`	orm	49	3.5	57	2.8
`objection`	orm	56	3	70	2.9
`mikroorm`	orm	108	4.1	121	4.1

45 Standalone cost of canonical response construction

The shared constructors enforce comparable output, but could themselves mask layer cost if they performed substantial graph work. Pattern by pattern, the point read copies seven declared fields and normalizes identifiers, views, the boolean, and timestamp; the range scan maps that same operation over the already ordered 20-row array without sorting; the deep-fetch graph path copies five post fields, one three-field author, and the already ordered comment array with each three-field author, while the flat-row variant performs the same field projection over rows already grouped by the adapter; and aggregation converts four scalar fields. The insert response does not use a graph constructor: each adapter returns only `{id: Number}`, and the out-of-band state check validates the persisted row. They only copy selected fields, normalize numbers, booleans, and ISO timestamps, and map arrays that the adapter has already grouped; they perform no query, join, regrouping, sorting, or cache access. Table S44 reports their isolated cost. Even the largest fixture is 0.146% of the smallest corresponding primary-campaign HTTP p50; the ten-comment deep-fetch forms are 0.072%. These are single-process development-host measurements and a scale comparison, not a decomposition of request time.

Table S44: Standalone cost of the shared canonical constructors on seed-realistic fixtures (30 timed blocks; 50,000 calls per block; median net cost after subtracting the identity-loop median). The p95-block column describes between-block variation. The final columns compare the constructor median with the smallest median HTTP p50 in the corresponding primary-campaign pattern, only as a scale reference. Database access, layer hydration, Express, and JSON serialization are excluded.

Operation	Fixture	Median (μs)	p95 block (μs)	Fastest HTTP p50 (ms)	Share
Point read	one seven-field row	0.84	0.85	7	0.012%
Range scan	20 rows	17.57	17.65	12	0.146%
Deep fetch (graph)	post + author + 10 comments	9.36	9.40	13	0.072%
Deep fetch (flat rows)	post + author + 10 comments	9.35	9.36	13	0.072%
Aggregation	four scalar fields	0.05	0.05	6	0.001%

46 Sensitivity of the matched-utilization capacity denominator

The utilization experiment used the 25-run median throughput at 50 connections as a predefined capacity proxy. Table S45 re-expresses the same offered loads against a separate empirical denominator: the maximum observed in the full concurrency sweep. The proxy is 93.2–102.1% of that maximum across the sixteen layer–engine cells, so the nominal 50% and 70% loads become 46.6–51.0% and 65.2–71.5%. This sensitivity analysis changes no p99 observation and keeps both interpretation bands below saturation.

Table S45: Sensitivity of the matched-utilization denominator on the deep fetch. The experiment set its capacity proxy $\widehat{C}_{50}$ to the median throughput of the 25-run, 50-connection primary cell. Treating its published within-campaign bootstrap interval as denominator uncertainty moves nominal 50% to 49.1–50.9% and nominal 70% to 68.8–71.2%. As a separately measured alternative, $\widehat{C}_{\text{sweep}}$ is the maximum throughput observed on the full concurrency ladder. The ratio $\widehat{C}_{50}/\widehat{C}_{\text{sweep}}$ spans 93.2–102.1%; consequently, loads labelled 50% and 70% under the original proxy correspond to 46.6–51.0% and 65.2–71.5% of the sweep maximum. Across the repeated sweep curves, the same nominal loads span 46.2–52.2% and 64.6–73.0%. This re-expression requires no new matched-tail run; both reported bands remain below saturation.

Engine	Layer	$\widehat{C}_{50}$	$\widehat{C}_{\text{sweep}}$	Ratio	nominal 50%	nominal 70%
PostgreSQL	`pg`	3638	3755	96.9%	48.4%	67.8%
PostgreSQL	`knex`	2431	2533	96.0%	48.0%	67.2%
PostgreSQL	`drizzle`	1829	1886	97.0%	48.5%	67.9%
PostgreSQL	`prisma`	1175	1180	99.6%	49.8%	69.7%
PostgreSQL	`typeorm`	1219	1281	95.2%	47.6%	66.6%
PostgreSQL	`sequelize`	978	1024	95.5%	47.8%	66.9%
PostgreSQL	`objection`	1017	1036	98.2%	49.1%	68.7%
PostgreSQL	`mikroorm`	519	533	97.4%	48.7%	68.2%
MySQL	`mysql2`	2416	2449	98.7%	49.3%	69.1%
MySQL	`knex`	1982	2071	95.7%	47.9%	67.0%
MySQL	`drizzle`	1301	1306	99.6%	49.8%	69.7%
MySQL	`prisma`	630	676	93.2%	46.6%	65.2%
MySQL	`typeorm`	1017	1038	98.0%	49.0%	68.6%
MySQL	`sequelize`	880	918	95.9%	47.9%	67.1%
MySQL	`objection`	836	819	102.1%	51.0%	71.5%
MySQL	`mikroorm`	472	474	99.6%	49.8%	69.7%

47 Same-host RQ2 campaign sensitivity

Table S46 compares the accepted primary campaign with a second independently ordered same-host campaign for the seven portable layers on deep fetch, aggregation, and insert. Both campaigns contain 25 complete repetitions per cell and zero timed request failures. The table reports exact-rank agreement, rank correlation, and maximum median drift. A reversal enters the main RQ2 result only when it recurs in both campaigns, both stack-specific gaps exceed 5%, and paired intervals exclude equality in opposite directions in both campaigns. This controls whole-campaign fragility on one host; it is not independent-host replication.

Table S46: Same-host, between-campaign sensitivity for the reviewer-prioritized RQ2 patterns. Both corrected-state campaigns use 25 repetitions per cell; the validation campaign uses a new block order and environment fingerprint. Leaders and Spearman rank agreement compare campaign medians for the seven portable layers. Exact ranks counts unchanged rank positions; drift is the largest absolute layer-level median-throughput change. A reversal is promoted in RQ2 only when it recurs, both stack-specific pair gaps exceed 5%, and paired-bootstrap intervals exclude equality in opposite directions in both campaigns. The margin predates these campaigns; the composite rule was specified after both completed and is applied uniformly to all pairs, so promotion is a post-hoc screen rather than a prespecified confirmatory test. Pairs meeting all criteria: Deep fetch: `prisma vs. objection`; Deep fetch: `prisma vs. sequelize`; Aggregation: `prisma vs. objection`; Insert: `prisma vs. mikroorm`. Campaign dates are not paired, and this is not cross-host replication.

Pattern	Stack	Primary leader	Validation leader	ρ	Exact ranks	max $\|\Delta\|$
Deep fetch	PG	`knex`	`knex`	1.00	7/7	2.3%
Deep fetch	MySQL	`knex`	`knex`	1.00	7/7	2.1%
Aggregation	PG	`drizzle`	`drizzle`	0.96	5/7	2.8%
Aggregation	MySQL	`mikroorm`	`mikroorm`	1.00	7/7	1.0%
Insert	PG	`knex`	`knex`	1.00	7/7	1.3%
Insert	MySQL	`drizzle`	`knex`	0.82	3/7	13.6%

48 Estimands and what the design does not identify

Table S47 consolidates, for each research question, the quantity the design estimates and the quantity it does *not* identify. It is reference material for reading the main text's claim limits in one place.

Table S47: Estimands defined in this study, consolidating the identification qualifications stated across the main text's Study Design and Results. Each quantity is *descriptive* of the treatment as defined; only relative within-condition differences are meant to travel, and rankings are configuration- and version-specific.

Estimand / quantity	**What it identifies**	**What it does *not* identify**	**Where reported**
Policy-selected documented configuration performance (RQ1, primary)	Performance of the configuration selected by the frozen-page policy (throughput, p99)	Library-only overhead; the most common or most performant usage	Main text, deep-fetch and insert tables
Common-SQL raw-path sensitivity contrast (diagnostic)	Residual spread once every layer runs common SQL through its raw facility	A causal decomposition or a bound on the library effect; the share from SQL formulation or query count	Supplement Table S42
Performance-conscious deep-fetch regime	Speed recoverable within a narrow byte-equivalent tuning budget (faster documented loading strategy)	Gains from SQL rewrites, caching, or schema change; non-drop-in strategies (excluded)	Supplement Table S18
Per-pattern capacity (estimated saturating throughput)	Each pattern's throughput knee per layer and engine, locating relative utilization at the operating point	Tail latency; a throughput quantity, not a demand- or utilization-dependent comparison on its own	Supplement Table S35, Figure S2
High-load p99 under equal closed-loop demand (primary tail)	The tail a deployment sees at fixed 50-connection demand, including acquisition queueing	The tail at comparable utilization; the pooled cross-run p99	Main text, deep-fetch table
Matched-utilization tail (p99 at matched fractions of own capacity)	p99 when each layer is offered 50% and most 70% of its estimated capacity; 85–95% cells are secondary near-saturation trends	The tail under equal external demand; a precise claim near 85–95% utilization	Supplement Tables S43, S45
Layer×engine interaction magnitude (RQ2)	The spread of each layer's PostgreSQL÷MySQL stack ratio and the concrete rank reversals	A pure DBMS effect; stability across hosts or deployments. Recurrence across the two same-host campaigns is used only as a within-host stability screen	Supplement Table S28; main text, Results (RQ2)

49 Protocol-development chronology

Table S48 dates each protocol element against the measurement campaigns, separating elements specified before any campaign from those added between campaigns and those introduced after the accepted campaigns. The last group was applied retrospectively, to archived state and the same deterministic seeded dataset, rather than gating the timed runs. It supports the development-versus-validation distinction drawn in Section 4.

Table S48: Chronology of protocol elements relative to the measurement campaigns. Dates are the first commit that added each artifact; campaign dates are from the archived environment captures (superseded pilot 2026-07-13, accepted corrected-state campaign 2026-07-24, same-host validation campaign 2026-07-25). Elements in the last group were introduced after the accepted campaigns and were therefore applied retrospectively, to archived state and to the same seeded dataset, rather than gating the timed runs prospectively. A first-commit date bounds when an element entered the recorded instrument; it is not evidence that the underlying idea was absent earlier.

Date	Stage	Protocol element	Relative to campaigns
2026-07-06	M1	Cross-implementation differential equivalence	Before any campaign
2026-07-06	R6	Resource accounting (CPU, RSS, GC)	Before any campaign
2026-07-11	R3	Common-SQL raw-path contrast	Before any campaign
2026-07-12	R4	Capacity characterization (warm-up/knee sweep)	Before any campaign
2026-07-14	R5	Operating-point separation (matched utilization)	Between campaigns
2026-07-20	M1	Post-write state validation	Between campaigns
2026-07-21	M1	Property-based read checks	Between campaigns
2026-07-22	–	Protocol formalized as machine-readable checklist	Between campaigns
2026-07-22	M2	Treatment-selection policy recorded	Between campaigns
2026-07-23	M2	Frozen documentation snapshots	Between campaigns
2026-07-26	M1	Specification-derived expected-result oracle	After accepted campaigns
2026-07-26	M1	Seed-parity verification	After accepted campaigns
2026-07-26	M1	Campaign-state verification	After accepted campaigns
2026-07-26	R7	Dead-work / implementation-waste check	After accepted campaigns

50 RQ2 leave-Prisma-out sensitivity

Table S49 repeats the RQ2 reversal-promotion procedure on the six portable layers other than Prisma, the one treatment whose lower-level MySQL driver differs. The promotion rule is unchanged. No pair is promoted, while cross-stack rank agreement moves in both directions: up on deep fetch and insert, down on aggregation. Holding out Prisma removes the driver substitution and the treatment together and therefore identifies neither as the cause.

Table S49: Leave-Prisma-out sensitivity for RQ2. Prisma is the only treatment whose lower-level MySQL driver differs from the rest, so the analysis is repeated on the six remaining portable layers under the identical promotion rule (recurrence in both corrected-state campaigns, both stack-specific gaps above the 5% margin, and paired-bootstrap intervals excluding equality in opposite directions in both campaigns). ρ is the PostgreSQL-versus-MySQL rank correlation of campaign medians. All 4 promoted reversals are lost when Prisma is held out, while cross-stack rank agreement moves in both directions: up on deep fetch and insert, down on aggregation, so agreement stays imperfect either way. Holding out Prisma removes the driver substitution and the treatment together and therefore identifies neither as the cause.

	All seven layers			Prisma held out (six layers)		
Pattern	ρ primary	ρ valid.	Promoted	ρ primary	ρ valid.	Promoted
Deep fetch	0.86	0.86	2	0.94	0.94	0
Aggregation	0.68	0.64	1	0.54	0.49	0
Insert	0.68	0.71	1	0.83	0.94	0

51 Protocol normative detail

This section carries the stage definitions, applicability limits, and compliance scheme referenced from Section 4.

Admission versus diagnosis Semantic admission asks two distinct questions. First, does an implementation satisfy expected results derived from the declared task and deterministic seed rather than from the timed native path? Second, are competing implementations mutually equivalent under the declared comparator? Failure of either read check, or a mutation reaching the wrong database state, excludes the cell. These are finite conformance tests, not proof of correctness for arbitrary inputs. Workload constraints disqualify only when preregistered; query count, joins, and loading strategy are otherwise measured properties of the treatment.

Applicability limits the output comparator must match the output semantics: byte-identical comparison, used here, is valid only when equivalent outputs are deterministic and canonically serializable, and otherwise (unordered collections, floats, timestamps, nondeterministic identifiers) needs a semantic comparator (set equality, tolerance, canonicalization). The experiment's outputs are deterministic and canonically constructed, so byte-equality is valid here.

Compliance levels M1 semantic admission and M2 treatment definition form the mandatory core because, without a declared task result and a stable configured treatment, there is no common comparison estimand. The remaining stages license additional interpretations without deciding whether a fixed-load result exists. Capacity characterization locates that result against a throughput knee. The common-SQL extension licenses a compound sensitivity contrast; utilization control licenses comparison at stated fractions of estimated capacity; resource accounting licenses a compute-and-memory reading or a labelled equal-budget sensitivity. Implementation-waste evidence records evidence against one class of avoidable work on the timed path, not its absence. Here M1–M2 cover the full matrix, R4 every pattern, R3/R5/R6 the deep fetch; R7 is met for the measured source by the static dead-work check, its runtime companion covering later campaigns only. Omitting a recommended stage narrows the supported claim without invalidating the reported numbers: Partial coverage of a recommended stage narrows what it licenses without withdrawing the admission evidence that licenses the comparisons.

Supplement Table S50 records how each control changed this experiment; that establishes consequence here, not universal necessity. Supplement Table S47 consolidates, per research question, the quantity estimated and the quantity not identified. Supplement Table S37 separately applies the codebook to nine external benchmark sources. The machine-readable audit preserves 63 source-located judgements, but one author performed the coding, so it demonstrates bounded protocol application, not inter-rater reproducibility. The record is structured for a second rater: additional codings attach alongside the author's, and a committed scorer (`npm run audit:agreement`) reports pooled percent agreement, Cohen's kappa with a bootstrap interval, and every disagreeing cell. Per-stage slices are n=9 with chance agreement up to 0.80, so per-stage kappa is flagged unreliable and the pooled figure is the headline. Result-blind forms for three optional independent exercises ship under `notes/reviewer-packets`; all remain pending, and the scorer reports agreement as not computable while one rater is recorded, so an unperformed exercise cannot be counted as validation.

52 Protocol-to-interpretation mapping

Table S50 records, per protocol stage, the concrete consequence it had for this study's reported interpretation. It is authored analytical reference material rather than a measurement table.

53 Prior-work coverage

Table S51 positions prior work along the four coverage axes this experiment spans: taxonomy breadth, engine coverage, joint throughput/tail reporting, and vendor independence. Each source covers at most two or three.

Table S50: The comparability protocol, stage by stage. Each row pairs a generic benchmarking principle with its access-layer manifestation, the risk if omitted, and how the stage affected interpretation here. The stages operationalize established principles; the single-rater external audit in Supplement Table S37 evaluates reported coverage without claiming priority. Throughputs abbreviated PG (PostgreSQL) and MySQL; "conns" denotes client connections.

Stage	Benchmark decision	Generic principle	Access-layer manifestation	Risk if omitted	Effect on interpretation here
(M1) Semantic admission	Which treatments are timed at all	Expected results and mutual equivalence precede speed	Access-layer paths can return different fields, scalar types, graph shapes, or mutation states; a fast cell may be a silent error doing less work	Wrong or incomplete implementations enter the ranking	The seed oracle supplied 73,080 expected-result checks; the admission layers caught multiple semantic and state defects, including shared MySQL contamination and unequal fan-out seed values
(M2) Treatment definition	Which API and loading strategy of each layer is measured	Pre-register and fix the system-under-test configuration reproducibly	Access layers expose multiple official query paths with different round-trip counts; native and builder facilities are fixed explicitly too	Undefined or outcome-informed API choice changes the estimand	Policy-selected documented deep fetch spreads 7.01× (PG) / 5.12× (MySQL); round-trips differ 1–4 by design; Drizzle is the documented tie-break case
(R3) Common-SQL raw-path sensitivity	Add a common-SQL raw-path contrast (a compound diagnostic, not a mechanism decomposition)	Standardize query text and plan while disclosing the jointly changed factors	Access-layer time mixes loading strategy with execution machinery; a common raw-path comparison shows sensitivity to standardization but does not separate those mechanisms	Mis-read the whole policy-selected documented spread as library-only "overhead"	The 7.01×/5.12× spread narrows to 1.71× (PG) / 2.03× (MySQL) among raw paths on common SQL under this compound raw-path sensitivity contrast; which component is responsible is not identified
(R4) Capacity characterization	Locate each layer's estimated saturating throughput before fixing an operating point	Interpret latency relative to capacity, not at an arbitrary fixed load	Configured implementations saturate at different throughputs, so fixed demand places them at different capacity fractions and queue depths	A fixed load compares layers at unequal utilization	A per-pattern sweep (1–200) puts every knee at or below 50 conns; median utilization 98–100% at the 50-conn point; native ∼7× the slowest ORM's capacity
(R5) Operating-point separation	Report tail at equal demand and at matched own-capacity fractions, as distinct quantities	Distinguish demand from utilization; correct coordinated omission	Because layer capacities differ, an equal-demand tail conflates queueing with sub-saturation latency; engines at similar capacity make the two nearly coincide	Read a capacity/queueing gap as a sub-saturation latency difference	At 50 conns deep-fetch p99 runs pg 18 ms to MikroORM 108 ms; at nominal 50% of the $\widehat{C}_{50}$ capacity proxy the p99 band contracts to 2.3–4.9 ms, and the full-sweep-denominator sensitivity stays below saturation (S45)
(R6) Resource accounting	Report performance with compute/memory demand or under an equal budget	Expose resource demand alongside speed	Similar throughput can consume different application or database CPU	A speed ranking hides its compute cost	CPU, RSS, requests per CPU-second, and an exploratory equal-CPU deep-fetch run qualify the throughput result
(R7) Implementation waste evidence	What evidence records that no treatment does avoidable work	Separate implementation quality from semantic conformance	Equivalent code can still allocate, convert, or serialize needlessly	Correct-but-inefficient adapter code is attributed to the library	Satisfied mechanically: the static dead-work check is clean on all 13 adapter modules of the measured source, so no cell is withdrawn on this ground. The stronger independent-human-review level is not claimed: all adapters and assignments are single-author, blind review packets are shipped, and this remains a validity threat

Table S51: Prior work on relational-database access-layer performance, positioned along four *coverage* axes this experiment spans; the paper's contribution is the comparability protocol of the main text, not this coverage. The main text cites each source by number. "Products covered" aggregates native drivers, query builders, and ORMs into one count; "none" marks studies that vary the engine or framework but not the access layer. *Vendor involvement* marks maintainer-authored studies — a conflict-of-interest property of the source, not a methodological guarantee that a non-vendor-authored benchmark is unbiased.

Study	Products covered	Engines	Metrics	Vendor involvement
Colley et al. (2018)	one ORM vs. hand-written SQL example	SQL Server	query latency	None
Procedia CS (2025)†	Prisma vs. raw SQL	PostgreSQL	latency, CPU	None
JCCT (2025)	3 ORMs	PostgreSQL	latency, memory, CPU	None
JCSI (2025)	3 ORMs	PostgreSQL	per-stage latency	None
Node-Bench (2023)	5 access products (+11 frameworks)	MySQL	throughput, latency	None
Salunke and Ouda (2024)	none (engine only)	PostgreSQL + MySQL	throughput, latency	None
Drizzle suite	9 products	PostgreSQL	throughput, latency, P95	Vendor
Prisma suite	3 ORMs	PostgreSQL	median latency only	Vendor
IMDBench	4 ORMs + driver	PostgreSQL	throughput, latency	Vendor
This study	**9 policy-selected documented (+2 tuned)**	**PostgreSQL + MySQL**	**throughput + response-time p99**	**None**

†Cited for its methodology only; its absolute speedup figures are not comparable to ours given the differing versions, hardware, workload, and harness.

54 Insert throughput and tail

Table S52 reports single-row insert throughput and response-time p99 per layer and engine under each engine's default durability, with within-campaign bootstrap intervals. The MySQL insert cells are the noisiest in the study, which is why no fine insert ordering is claimed.

55 Result-graph breadth sweep

Table S53 varies the deep fetch's result-graph breadth from 0 to 500 child rows and reports each layer's throughput multiplier relative to its engine's native driver. The multipliers are near-flat on MySQL but not on PostgreSQL, where the native-relative spread widens from 6.0× to 11.7×: MikroORM and Sequelize fall further behind the native driver as breadth grows, while Knex and Drizzle close on it. Breadth is the only dimension varied, so the sweep bounds neither graph depth, nor schema, nor query mix, and three runs per cell make it exploratory rather than an estimate.

56 Multiplicity and exchangeability in the paired analyses

Multiplicity across the adjacent-pair families Two further test families are reported besides the RQ2 promotion screen and its correction (Section 58): the seven adjacent throughput pairs on MySQL (Table S36) and

Table S52: Insert: throughput (req/s, higher is better; median with a within-campaign 95% bootstrap interval over the 25 repeated runs — run-to-run variability within the source campaign of each cell, not across hardware or days) and response-time p99 (ms, lower is better; median run-level p99 with the same within-campaign 95% bootstrap interval) by access layer and engine. p99 is reported at 1 ms resolution, so cells differing by ≤ 1 ms are practically unresolved (see the paired significance analysis in the supplement).

		PostgreSQL		MySQL	
Layer	Category	req/s [95% CI]	p99 [95% CI]	req/s [95% CI]	p99 [95% CI]
pg	native-driver	6018 [5847–6185]	12 [11–13]	–	–
mysql2	native-driver	–	–	1766 [1451–1844]	116 [105–127]
pg-tuned	native-tuned	6125 [5868–6254]	12 [11–14]	–	–
mysql2-tuned	native-tuned	–	–	1683 [1496–1797]	117 [106–124]
knex	query-builder	4581 [4490–4715]	16 [14–18]	1779 [1418–1806]	118 [105–125]
drizzle	orm	4091 [4010–4163]	15 [15–16]	1816 [1624–1837]	105 [98–114]
prisma	orm	2695 [2675–2740]	24 [23–25]	894 [859–912]	129 [116–144]
sequelize	orm	2616 [2564–2649]	23 [22–24]	1503 [1430–1696]	119 [108–134]
typeorm	orm	2567 [2526–2602]	25 [24–26]	1355 [1248–1388]	115 [108–117]
objection	orm	3780 [3686–3852]	17 [16–20]	1807 [1514–1818]	108 [101–118]
mikroorm	orm	1894 [1848–1915]	32 [31–34]	1237 [1103–1282]	110 [102–114]

the fourteen adjacent p99 pairs, seven per engine (Table S30). Both follow the *observed* ranking rather than a preregistered one, so they are secondary descriptive robustness checks and not a confirmatory family. As a conservatism check we apply a Bonferroni correction: at $\alpha = 0.05$ the per-comparison threshold is $0.05/7 = 7.1 \times 10^{-3}$ for the throughput family, $0.05/14 = 3.6 \times 10^{-3}$ for the p99 family, and $0.05/21 = 2.4 \times 10^{-3}$ if the two are pooled. All seven throughput comparisons and thirteen of the fourteen p99 comparisons sit at the permutation resolution floor $1/(B+1) = 5.0 \times 10^{-5}$ with $B = 20{,}000$ and survive every threshold. One does not: the MySQL Sequelize-against-Objection p99 step has $p = 0.0223$, which clears an uncorrected $\alpha = 0.05$ but fails all three corrected thresholds. That adjacent step is therefore not resolved once multiplicity is accounted for, and we do not read an ordering from it. We state this rather than omit it because the absence of a multiplicity policy, not its outcome, is what a reader cannot check. The floor also means the tests bound the p-value from above and do not resolve differences among the pairs: a comparison at the floor is not evidence that one adjacent step is more separated than another, for which the ratio and its interval are the informative quantities.

Exchangeability of the replicate blocks The paired analyses treat the 25 replicates as exchangeable blocks: within each replicate every layer is measured once, from the same seeded identifier distribution, and the layer order is randomized. The permutation tests permute layer labels within blocks, which is valid under the null if block position carries no systematic effect.

Table S53: Native-relative throughput multiplier on the deep fetch as the result-graph breadth varies from 0 to 500 child rows (median of 3 runs per cell, exploratory). A value of 1.0 is parity with the engine's native driver; larger is slower. The multipliers are near-flat on MySQL, where the native-relative spread moves 3.9× to 4.0×, but not on PostgreSQL, where it widens 6.0× to 11.7×: MikroORM and Sequelize diverge from the native driver as breadth grows while Knex and Drizzle converge toward it. The sweep varies breadth only, not graph depth, schema, or query mix, so it bounds no other dimension.

Layer	0	1	10	50	100	500
PostgreSQL						
`knex`	1.5	1.7	1.5	1.3	1.2	1.1
`drizzle`	2.2	2.3	2.1	1.6	1.4	1.1
`prisma`	3.0	3.2	3.1	2.8	2.7	2.6
`sequelize`	3.2	3.4	3.6	4.1	4.4	4.9
`typeorm`	3.0	3.1	3.1	3.3	3.2	3.7
`objection`	3.1	4.5	3.8	3.0	2.7	2.4
`mikroorm`	6.0	6.8	7.3	8.6	9.6	11.7
MySQL						
`knex`	1.3	1.2	1.2	1.2	1.2	1.2
`drizzle`	1.9	1.9	1.9	1.9	1.9	1.9
`prisma`	3.9	3.6	3.6	3.4	3.7	3.7
`sequelize`	2.2	2.2	2.2	2.1	2.3	2.3
`typeorm`	2.3	2.4	2.4	2.3	2.6	2.5
`objection`	2.4	2.4	2.5	2.4	2.4	2.4
`mikroorm`	3.7	3.8	3.8	3.7	4.0	4.0

That assumption is checked directly only for the write path, where an explicit forward/reverse order-invariance run is reported; for the read workloads it is supported by design (randomized order, per-replicate reset, handler drain between cells) and by the low within-campaign dispersion (deep-fetch CV $\leq 4.6\%$), but no separate block-order or time-trend diagnostic is displayed. Database and host state persist across a campaign, so a slow drift shared by all layers would cancel in the paired contrasts while a layer-specific drift would not. The paired intervals and tests therefore rest on an unchecked exchangeability assumption for the read workloads, which we state as a limitation rather than treat as established.

57 Effect sizes for the promoted cross-stack reversals

Table S54 sizes the four promoted reversals that the main text's RQ2 answer rests on. Each is a paired within-campaign ratio with a 95% bootstrap interval on each stack, so the reversal is visible as the two intervals falling on opposite sides of parity rather than as a rank statement alone. The largest is the insert reversal (PostgreSQL 1.423 against MySQL 0.723) and the smallest the deep-fetch pair against Objection (1.155 against 0.754). The MySQL insert interval is the widest in the table, consistent with that cell's 13.6% within-campaign dispersion, which is why no fine MySQL insert ordering is claimed anywhere. These intervals are within-campaign: they cover replicate-to-replicate variation on one host and one software freeze, not cross-host or cross-version transfer.

58 Familywise correction across the RQ2 screen

The promotion rule is applied to 21 layer pairs on each of three patterns, 63 comparisons per campaign, without a familywise correction, and the main text states that recurrence across two same-host campaigns does not bound familywise error. Table S55 supplies the correction that disclosure leaves open. The corrected hypotheses are the necessary components of what the rule claims, which is a conjunction: the pair's ratio differs from 1 on both stacks, in opposite directions, by more than the margin. Testing the cross-stack interaction instead would certify a hypothesis the rule already implies, since two intervals on opposite sides of parity entail a non-zero interaction, and so would not cover the selection. Each stack's per-replicate log ratios are therefore tested against zero by two-sided sign-flip permutation, the two are combined by intersection-union (the larger of the two p-values, the correct combination for a conjunction; using two-sided components for a directional claim is conservative), and the result is Holm-corrected across all 63 pair-pattern screens in a campaign. Holm retains 53 of 63 in the primary cam-

Table S54: Effect sizes for the 4 promoted cross-stack reversals. Each row gives the ratio of campaign median throughputs, first layer over second, with a paired bootstrap 95% interval, on each stack. This estimator differs slightly from the geometric mean of per-replicate ratios used in Tables S28 and S36; the promotion decision is unaffected. A pair is promoted only when the two stacks place it on opposite sides of parity, both intervals exclude 1.0, and the reversal recurs in the validation campaign. Every promoted pair involves Prisma, whose MySQL path uses the MariaDB adapter rather than `mysql2`, so the design cannot attribute these reversals to the DBMS rather than to the adapter, the driver, or Prisma's own implementation. The intervals are within-campaign and do not cover cross-host or cross-version variation.

		PostgreSQL		MySQL	
Pattern	Pair (A vs B)	A/B	95% CI	A/B	95% CI
Deep fetch	`prisma` vs `objection`	1.155	[1.137, 1.164]	0.754	[0.743, 0.765]
Deep fetch	`prisma` vs `sequelize`	1.201	[1.185, 1.208]	0.716	[0.710, 0.725]
Aggregation	`prisma` vs `objection`	1.163	[1.140, 1.176]	0.806	[0.798, 0.818]
Insert	`prisma` vs `mikroorm`	1.423	[1.405, 1.466]	0.723	[0.682, 0.804]

paign and 50 of 63 in the validation campaign. Applying surrogate direction and margin filters to the survivors, computed on geometric means rather than the rule's medians, leaves exactly the four pairs the screen promoted, and in the validation campaign those four plus one that the recurrence requirement excludes; that agreement is descriptive concordance between two estimators, not evidence that the promotion decision itself was corrected. The two estimators are close here: for all four promoted pairs on both stacks and in both campaigns they agree within 2.2%, and each pair's median-ratio bootstrap interval excludes the $\pm 5\%$ band on both stacks in both campaigns. The sign-flip null here is symmetry of the per-replicate log ratios about zero, a different and equally unchecked assumption from the within-block label exchangeability of Section 56. What this does and does not settle is worth stating plainly: it shows that the per-stack nonzero-effect evidence for the four promoted pairs survives Holm correction, and it leaves untouched the two limitations the main text already reports: the 5% margin was fixed before both campaigns, but the composite direction-and-margin rule that uses it was specified after they completed, and the two campaigns share a host and the same seeded draws. Correction cannot make a post-hoc screen prospective.

Table S55: Familywise correction of the per-stack nonzero-effect evidence behind the promoted reversals. Promotion requires a *conjunction*: the pair's ratio differs from 1 on PostgreSQL and on MySQL, in opposite directions, by more than the margin. The hypotheses corrected here are the necessary components of that conjunction, one per stack: each stack's per-replicate log ratios are tested against zero by two-sided sign-flip permutation ($B = 20000$, per-test seeds derived from campaign, pattern, pair and engine), the two combined by intersection-union (the larger p), and the result Holm-corrected across all 63 pair-pattern screens in a campaign. What is *not* corrected is the rest of the rule: its direction and margin criteria are evaluated on ratios of campaign medians with paired bootstrap intervals, and none of that enters the corrected family. Holm retains 53 of 63 screens in the primary campaign and 50 of 63 in the validation campaign. Applying surrogate direction and margin filters to the survivors, computed on geometric means rather than the rule's medians, leaves exactly the 4 pairs the screen promoted in the primary campaign, and those 4 plus `prisma` versus `sequelize` on insert in the validation campaign, which the recurrence requirement excludes. That agreement is descriptive concordance between two estimators, not evidence that the promotion decision itself was corrected. Two limits. Each promoted p is at the permutation resolution floor $1/(B{+}1) = 5.0{\times}10^{-5}$, sixteen-fold below the strictest Holm threshold 0.05/63, so surviving correction follows from the floor rather than being tested by it. And correction does not make the screen prospective: the 5% margin was fixed before both campaigns, but the composite rule using it was specified after they completed, and the two campaigns share a host and the same seeded draws.

Pattern	Promoted pair	primary p	validation p	Holm
Deep fetch	`prisma` vs `sequelize`	$5.0{\times}10^{-5}$	$5.0{\times}10^{-5}$	yes
Deep fetch	`prisma` vs `objection`	$5.0{\times}10^{-5}$	$5.0{\times}10^{-5}$	yes
Aggregation	`prisma` vs `objection`	$5.0{\times}10^{-5}$	$5.0{\times}10^{-5}$	yes
Insert	`prisma` vs `mikroorm`	$5.0{\times}10^{-5}$	$5.0{\times}10^{-5}$	yes

59 Equivalence assessment for the closest adjacent pair

Table S56 reports the ±5% equivalence assessment the main text refers to. Three properties bound how it may be read. First, the assessment is the confidence-interval form of TOST: it asks whether the 90% paired bootstrap interval of the geometric-mean ratio lies inside the equivalence bounds, and reports no separate one-sided test statistics. Second, a negative result means equivalence within the margin was *not established*; it does not reject equivalence, and here neither interval lies wholly outside the margin either, since both straddle the upper bound. Third, the pair is selected from the observed ordering as the smallest adjacent ratio among the eight policy-selected layers on that engine, so the selection is post hoc and the assessment is a sensitivity analysis rather than confirmatory evidence. The margin shares that status: ±5% is author-selected and application-dependent rather than a universal threshold, and although it was fixed on 2026-07-13, before the accepted campaigns, pilot data already existed at that date, so it is not a preregistered bound. The tuned baselines are excluded because the paper defines them as disclosed reference points rather than treatments; the closest adjacent pair including them would be `pg-tuned` versus `pg`, which does satisfy the margin.

Table S56: Equivalence assessment for the closest adjacent policy-selected pair on the deep fetch, one pair per engine, over the 25 paired replicates. The pair is chosen from the data as the smallest adjacent ratio of cell medians among the eight policy-selected layers on that engine, so the selection is post hoc; the tuned baselines are excluded as disclosed reference points. The procedure is the confidence-interval form of TOST: the 90% paired bootstrap interval of the geometric-mean ratio must lie inside the ±5% equivalence bounds $[0.9524, 1.0500]$. Neither interval does, so equivalence within ±5% is *not established* for either pair. This is not the same as rejecting equivalence, and neither interval places the difference wholly outside the margin either: both straddle the upper bound. Seed `0x57a75`, $B = 20000$ resamples.

Engine	Closest adjacent pair	n	ratio	90% CI	equiv.
PostgreSQL	`typeorm` vs `prisma`	25	1.0457	[1.0339, 1.0577]	no
MySQL	`sequelize` vs `objection`	25	1.0572	[1.0450, 1.0699]	no